Academic Project Report

Autonomous Pick-and-Place with the Franka FR3

Authors : Dmytro SHLIAHA & Anna SOBCHUK

Spring 2026

Abstract

This report presents the design, implementation, and evaluation of an autonomous pick-and-place system built around a Franka FR3 seven-degree-of-freedom robot arm. The objective was to develop a pipeline capable of detecting a metal food can placed at an arbitrary position on a table surface, computing its three-dimensional position from camera data, executing a reliable grasp, and depositing the can behind the robot — all without human intervention and within a cluttered real laboratory environment.

The system integrates three principal subsystems: a dual computer-vision pipeline combining YOLOv8n object detection with a HoughCircles fallback, operating on colour and depth streams from an Intel RealSense D435 camera mounted on a custom aluminium profile frame; a direct regression-based coordinate mapping that translates camera-frame detections into robot-base coordinates without a TF publisher node; and a motion-control layer built on FrankaPy with direct ROS `actionlib` gripper control. The entire software stack runs inside a Docker container that encapsulates ROS Noetic, the `franka-interface` action server, and all Python dependencies.

The principal engineering challenges addressed include the real-time kernel requirements of the Franka Control Interface, a namespace mismatch that caused FrankaPy's built-in gripper methods to hang indefinitely, the difficulty of detecting a metallic reflective can with a COCO-pretrained model in a cluttered scene, the fragility of geometry-based extrinsic camera calibration, and workspace constraints that caused naive single-step joint rotation to fail. Each challenge was resolved through a concrete engineering solution: a two-step joint-space rotation via a safe centred intermediate configuration, an empirical regression mapping fitted on four physical calibration points, a HoughCircles fallback detector with CLAHE preprocessing, and a run-time gripper width verification that detects missed grasps before any rotation begins.

The resulting system reliably detects cans across a cluttered workspace, completes the full pick-rotate-place cycle in approximately 35 seconds per run, and operates continuously over multiple consecutive cycles via an arm/disarm synchronisation mechanism, as demonstrated by repeated successful runs in the laboratory. A purpose-built passive telemetry node, `session_monitor.py`, observes the pipeline through read-only subscriptions, publishes per-cycle timing and grasp quality metrics, and provides an automatic watchdog that recovers from stuck-picker failures without modifying any application code.

The complete source code repository containing the implementation of the project is available on GitHub [1].

Keywords: Franka FR3, pick-and-place, ROS Noetic, FrankaPy, YOLOv8, HoughCircles, Intel RealSense D435, coordinate regression, Docker, robot manipulation, session monitor, watchdog, telemetry.

Contents

Abstract **1**

Contents **4**

List of Figures **6**

List of Tables **6**

1 Introduction **7**
1.1 Motivation and Context . . . 7
1.2 Project Objectives . . . 8
1.3 Scope and Assumptions . . . 8
1.4 Structure of the Report . . . 8

2 Review of Literature **9**
2.1 Robot Manipulation Pipelines . . . 9
2.2 Object Detection for Manipulation . . . 9
2.3 Depth Sensing and 3D Back-Projection . . . 10
2.4 Hand-Eye Calibration . . . 10
2.5 Motion Planning for Robot Arms . . . 10
2.6 The ROS Ecosystem and Docker Deployment . . . 10

3 Hardware and Software Stack **11**
3.1 The Franka FR3 Robot Arm . . . 11
3.2 The Franka Hand Gripper . . . 13
3.3 Intel RealSense D435 Depth Camera . . . 14
3.4 Software Stack Overview . . . 14
3.4.1 FrankaPy . . . 14
3.4.2 Docker Container Configuration . . . 16
3.4.3 Real-Time Kernel Requirement . . . 16
3.4.4 Python Dependencies . . . 17

4 Object Detection Pipeline **18**
4.1 Top-Down Workspace View . . . 18
4.2 Dual Detection Strategy: YOLO and HoughCircles . . . 18
4.3 YOLOv8 Integration . . . 19
4.3.1 Confidence Threshold and Class Selection . . . 20
4.3.2 Geometric Diameter Filter . . . 20
4.4 HoughCircles Fallback . . . 21
4.5 Depth Back-Projection . . . 22
4.6 Stability Buffer . . . 22
4.7 Detection in a Cluttered Environment . . . 23

5 Coordinate Frame Calibration **25**
5.1 The Camera-to-Robot Transform Problem . . . 25
5.2 Approach 1: ROS TF Static Transform . . . 25

5.3 Approach 2: Regression-Based Mapping . . . 26
5.3.1 Calibration Data Collection . . . 26
5.3.2 Linear Regression Model . . . 26
5.4 Workspace Validity Check . . . 27
5.5 Elimination of the TF Publisher Node . . . 27
5.6 Recalibration Procedure . . . 27

6 Robot Motion Control 28
6.1 The Manual Baseline Script . . . 28
6.2 Gripper Control . . . 28
6.2.1 Namespace Mismatch and the Direct Actionlib Solution . . . 28
6.3 The Two-Step Rotation Problem and Solutions . . . 29
6.3.1 Initial Failure: Single-Joint Rotation . . . 29
6.3.2 Manual Test Solution: Via 90-Degree Midpoint . . . 30
6.3.3 Autonomous Picker Solution: Via Safe Centred Configuration . . . 31
6.4 Gripper Tilt for Extended Reaches . . . 32
6.5 The Full Pick-and-Place Sequence . . . 32

7 System Integration 34
7.1 ROS Node Architecture . . . 34
7.2 Inter-Node Communication Protocol Design . . . 34
7.3 Arm/Disarm Synchronisation . . . 35
7.4 Fault Tolerance and Exception Safety . . . 36
7.5 Containerisation and Reproducible Deployment . . . 38

8 Program Control Flow 39
8.1 System Data-Flow Topology . . . 39
8.2 Detection Pipeline . . . 39
8.3 Pick-and-Place Sequence . . . 43
8.4 Known Corrections to the Diagrams . . . 45

9 Results and Evaluation 47
9.1 Multi-Cycle Autonomous Operation . . . 47
9.2 Cycle Time . . . 48
9.3 Detection Reliability . . . 48
9.4 Grasp Reliability . . . 48
9.5 Fault Tolerance in Practice . . . 49

10 Session Monitor Node 50
10.1 Observability in ROS-Based Robot Systems . . . 50
10.2 Architectural Design: The Passive Observer Pattern . . . 50
10.3 Event-Driven State Inference . . . 51
10.4 Grasp Quality Metric . . . 51
10.5 Camera Frame Rate Estimation . . . 53
10.6 Watchdog and Fault Recovery . . . 53
10.7 Published Metrics . . . 53

11 Conclusions and Future Work **55**
11.1 Summary of Contributions 55
11.2 Lessons Learned 56
11.3 Directions for Future Work 56
11.3.1 Custom Detection Model 56
11.3.2 Proper Hand-Eye Calibration 56
11.3.3 Visual Servoing 57
11.3.4 Extended Session Monitor and ROS 2 Migration 57
11.3.5 Towards a Mobile Trash-Picking Robot 57
11.4 Final Remarks 57

Bibliography **59**

List of Figures

1 Full laboratory setup. The Franka FR3 arm is mounted on the table (left). The aluminium profile frame structure extends to the right, with the Intel RealSense D435 camera clamped to its horizontal cross-beam and aimed downward toward the workspace. A metal food can is placed on the table surface within the camera's field of view. 12

2 The Franka FR3 arm in the laboratory in its home/ready configuration. The arm's characteristic seven-joint profile is visible. The blue LED ring at the base indicates the arm is active and the brakes are released. The aluminium frame and camera stand are visible in the background. 13

3 Official Franka system architecture diagram. The PC workstation running ROS and `libfranka` communicates with the arm through the Franka Control Interface (FCI) box via Ethernet. The External Activation Device (EAD) must remain pressed to keep the brakes released during operation. The Emergency Stop provides a hardware safety cutout. 13

4 Intel RealSense D435 RGB-D camera and its mounting configuration on the custom aluminium frame structure. 15

5 Top-down view of the workspace from the RealSense camera's perspective. The robot base is visible at the top of the frame; the metal food can is placed on the wooden table surface. The RealSense camera is visible at the bottom of the image in its frame-mounted position. The tape markers on the table are calibration reference positions used during the coordinate mapping. 19

6 Debug detection output (`/object_detector/yolo_debug` topic). The workspace is viewed from above. The metal can is detected and highlighted with a bounding box; the estimated diameter and depth ($D = 7.8$ cm, $Z = 0.70$ m) are shown alongside the detection method label (YOLO:`cup` and Hough both active). The red dot marks the sampled 3D back-projection point. The "ROBOT BUSY" overlay in the top-left corner indicates the picker is executing a motion and the detector is in its disarmed state. Despite the visible clutter in the environment — including the robot base itself and other objects — the system reliably focuses on the can. 24

7 Two key stages of the pick-and-place sequence. Left: can securely gripped and lifted. Right: arm rotated 165° and can positioned above the placement point behind the robot. 32

8 Completion of the pick-and-place cycle. Left: arm lifting away after placing the can. Right: arm returned to the home configuration, detector re-armed. 33

9 System data-flow topology. The Intel RealSense D435 publishes colour and depth image streams into `object_detector.py`. A confirmed can detection is forwarded to `franka_picker.py` on `/detected_object/point`. The picker sends motion commands directly to the Franka FR3 arm via FrankaPy and to the Franka Hand via `actionlib` goal messages. The picker publishes `Bool(True)` on `/franka_picker/done` to re-arm the detector for the next cycle. The passive `session_monitor.py` node observes both event topics without publishing to them and exports derived metrics to the `/session_monitor/*` namespace. . . 40

10 Detection pipeline control flow (`color_cb()` in `object_detector.py`, executed at 15 fps). Both detection branches run on every armed frame. The YOLO branch passes low-confidence detections (conf = 0.10) through a diameter size filter and 3D back-projection. The HoughCircles branch first enhances contrast with CLAHE and applies Gaussian blurring before circle detection, then applies the same size filter and back-projection. The resulting candidates are merged (YOLO preferred), the closest candidate is selected, and the stability buffer accumulates consistent detections over 8 consecutive frames before a `PointStamped` is published. 42
11 Pick-and-place sequence control flow (`point_cb()` in `franka_picker.py`). Steps 1–11 form the nominal success path (centre column). Three embedded verification checks guard against descent failures, missed grasps, and in-transit drops (right column abort paths). All paths, nominal and abort, converge into the `finally` block, which guarantees that `/franka_picker/done` is published regardless of how the sequence terminates. 44
12 Terminal output from four consecutive autonomous pick-and-place cycles in a single laboratory session. Each cycle begins when the detector publishes a confirmed can position and ends when the arm returns home and publishes `Bool(True)` to `/franka_picker/done`. The session monitor logged a mean cycle duration of approximately 35 seconds with a spread of less than 3 seconds across the four runs, confirming consistent motion execution. 47

List of Tables

1 Franka FR3 joint limits. 11
2 Gripper parameters used for metal can manipulation. 14
3 Software stack components and their execution context. 15
4 Pick-and-place sequence in `franka_picker.py`. 33
5 ROS topics used by the system. 34
6 Topics published by `session_monitor.py` to the `/session_monitor/` namespace. 54

1 Introduction

Robotic manipulation of everyday objects represents one of the most active and practically significant areas of modern robotics research. While industrial robots have performed highly repetitive pick-and-place tasks for decades in structured factory environments, enabling a robot to reliably grasp an arbitrary object placed at an unknown position in a semi-structured workspace remains a genuinely difficult engineering problem. The difficulty arises not from any single hard challenge but from the tight coupling of several subsystems (e.g. perception, coordinate transformation, motion planning, and physical actuation) each of which must function correctly for the end-to-end task to succeed.

This report documents the design and implementation of an autonomous pick-and-place system built around a **Franka FR3** seven-degree-of-freedom (7-DOF) collaborative research arm. The system was developed as part of an academic project at ENSTA Paris with the goal of demonstrating fully autonomous manipulation of metal food cans: the robot must detect a can placed anywhere within a reachable area of a table surface, compute its position in three-dimensional space, grasp the can reliably, carry it through a rotation of approximately 165 degrees, and deposit it on the table behind the robot — all without human intervention. A key requirement of the demonstration environment was that the system must function correctly even when many unrelated objects are visible in the camera's field of view, as is typical in a real laboratory setting.

1.1 Motivation and Context

Research-grade robot arms such as the Franka FR3 occupy an important niche in the robotics landscape: they are safe enough to operate in proximity to humans, precise enough for fine manipulation tasks, and open enough in their software interfaces to support rapid academic prototyping. The FR3's Franka Control Interface (FCI) exposes real-time joint-level control at 1 kHz, making it possible to implement custom controllers that go well beyond the pre-packaged trajectories of most industrial systems.

At the same time, using the FR3 effectively requires navigating a software stack of considerable depth. The real-time communication protocol imposes strict kernel constraints; the recommended Python abstraction library (FrankaPy) diverges in important ways from the MoveIt-based workflow described in most online tutorials; and the Docker-based deployment environment introduces its own layers of configuration complexity. A significant portion of this project's effort was spent understanding, debugging, and working around this stack — producing lessons that are directly transferable to future projects with the same or similar hardware.

The choice of metal cans as the manipulation target is motivated by an autonomous waste-collection application context. The long-term vision behind this project is a system capable of identifying litter, picking it up, and depositing it in a bin, with metal cans as a first and representative test object. Cans are also, paradoxically, among the more difficult everyday objects for a depth-camera vision system to handle, because their metallic surfaces produce specular reflections that confuse both RGB-based detectors and structured-light depth sensors. Making the system work reliably for this challenging object class in a cluttered environment therefore provides a solid foundation for broader manipulation applications.

1.2 Project Objectives

The primary objective of this project was to develop a complete, end-to-end autonomous pick-and-place pipeline. The specific requirements were:

- Detect the presence of a metal can anywhere within a designated workspace area, in the presence of many unrelated objects, without prior knowledge of the can's position.
- Estimate the can's three-dimensional position with sufficient accuracy for the gripper to make reliable contact.
- Execute a safe grasp that holds the can firmly without crushing it.
- Transport the can through approximately 165 degrees of base rotation and deposit it on the table behind the robot.
- Return the arm to a home position and automatically re-arm for the next detection cycle, enabling continuous multi-run operation.
- Package the entire system in a reproducible, containerised deployment environment that is portable across different host machines.

A secondary objective was to document the implementation process in sufficient detail that future students working with the same hardware can build on this work rather than rediscovering its pitfalls from scratch.

1.3 Scope and Assumptions

The work presented here is bounded by several deliberate constraints. The robot operates with the standard Franka Hand parallel-jaw gripper; no custom end-effector was designed. The vision system is restricted to a single, statically mounted Intel RealSense D435 depth camera, fixed to a custom aluminium profile frame above the table. The primary detection model is YOLOv8n trained on COCO, augmented by a HoughCircles fallback for metallic objects that the neural detector misses. The workspace is a flat wooden table in a real laboratory environment containing many other objects visible to the camera. No assumptions are made about can position, but the system handles a single can at a time.

1.4 Structure of the Report

The remainder of this report is organised as follows. Section 2 reviews the relevant literature on robot manipulation, object detection, depth sensing, hand-eye calibration, and the ROS ecosystem. Section 3 describes the hardware platform and the complete software stack. Section 4 presents the dual detection pipeline including the YOLOv8 integration, the HoughCircles fallback, depth back-projection, and the stability buffer. Section 5 describes the coordinate calibration methodology and the empirical regression mapping. Section 6 covers all aspects of robot motion control. Section 7 presents the system integration, the ROS node architecture, and the inter-node communication design. Section 8 consolidates the runtime behaviour of all three principal program control flows into annotated flowcharts with corresponding code extracts. Section 9 presents the experimental results and evaluates system performance. Section 10 describes `session_monitor.py`, a purpose-built passive telemetry and watchdog node. Section 11 draws conclusions and identifies directions for future work.

2 Review of Literature

This section surveys the scientific and engineering literature that informed the design choices made in this project. The relevant fields span robot manipulation pipelines, deep learning-based object detection, classical computer vision, depth-sensor integration, hand-eye calibration, and the ROS software ecosystem.

2.1 Robot Manipulation Pipelines

The canonical architecture for robot manipulation can be decomposed into four tightly coupled stages: *perception*, *grasp planning*, *motion planning*, and *execution* [2]. Each stage introduces its own failure modes, and the challenge of building a robust end-to-end system lies as much in the interfaces between stages as in the individual algorithms.

Early pick-and-place systems relied almost exclusively on structured environments: known object geometry, fixed lighting, and fiducial markers or part feeders that constrained object position to a small set of known poses. Such systems remain dominant in industrial applications where throughput and repeatability outweigh flexibility [3]. Research systems have progressively relaxed these assumptions, moving toward bin-picking of randomly posed objects and eventually toward the fully unstructured grasping of novel objects [4].

The Franka FR3 (and its predecessor, the Franka Emika Panda) has become a widely used research platform precisely because its 7-DOF kinematic structure provides redundancy that simplifies collision avoidance in cluttered workspaces, while its torque-controlled joints enable compliant manipulation strategies beyond simple position control [5]. Several benchmark manipulation datasets and challenge tasks have been defined around this arm family, making it straightforward to compare new results against existing literature.

2.2 Object Detection for Manipulation

Object detection for robot manipulation has been transformed by the development of deep convolutional neural networks. The YOLO (You Only Look Once) family of detectors [6] established a paradigm in which a single neural network predicts bounding boxes and class probabilities directly from full images in a single forward pass, making near-real-time detection feasible on commodity GPU hardware. The YOLOv8 architecture used in this project represents the current state of the art from Ultralytics [7]. The nano variant (YOLOv8n), chosen for its inference speed, delivers competitive accuracy on the 80-class COCO dataset [8] with a model size of approximately 3 MB.

An important limitation for manipulation tasks is that the COCO dataset does not contain a dedicated class for metal food cans. The closest available proxy classes (bottle, wine glass, cup) are insufficient for highly reflective metallic surfaces. This fundamental gap between training distribution and deployment conditions is a well-known problem in applied computer vision [9], and it motivates the use of a classical fallback detector in this project.

Classical circle detection via the Hough transform [10] provides an alternative that is independent of learned appearance models. By detecting circular edges rather than object categories, HoughCircles can identify cylindrical cans from any viewpoint that presents a circular or elliptical profile. Its principal sensitivity is to scene contrast rather than learned semantics, making it complementary to the YOLO-based approach and a natural fallback when the neural detector fails on atypical objects.

2.3 Depth Sensing and 3D Back-Projection

Recovering the three-dimensional position of a detected object from a single camera requires either depth information or geometric priors. The Intel RealSense D435 used in this project is a stereo-based active sensor that provides aligned colour and depth streams [11]. Its key practical advantages are its compact form factor, availability as a ROS package, and factory-calibrated intrinsics accessible via the `camera_info` topic. Its principal limitation is the difficulty of obtaining reliable depth readings from highly reflective surfaces: specular highlights cause the depth image to return zero or NaN readings precisely at the centre of a metallic can. The sampling strategy used in this project to mitigate this is described in Section 4.

2.4 Hand-Eye Calibration

Hand-eye calibration determines the geometric transformation between a camera and the robot base or end-effector [12, 13]. Classical approaches require the robot to observe a calibration target from multiple poses and solve a matrix equation of the form $AX = XB$ [14]. In practice these methods are sensitive to angular estimation errors, which compound with scene depth and can translate centimetre-scale offset errors at the camera into gripper misses at the table. For constrained workspaces, an empirical regression approach — fitting a linear mapping from a small set of correspondence measurements — can provide comparable accuracy with less fragility [15]. This approach was adopted after the classical TF-based method proved insufficiently robust, as described in Section 5.

2.5 Motion Planning for Robot Arms

Motion planning for redundant manipulators involves finding a collision-free, kinematically feasible path through joint space [16]. FrankaPy [17], the control library used in this project, provides a high-level Python API for Cartesian and joint-space motions executed directly on the Franka's internal controller, bypassing MoveIt entirely. This makes it convenient for well-defined motions but requires careful manual waypoint engineering to avoid singularities and workspace limits, a challenge addressed in Section 6.

2.6 The ROS Ecosystem and Docker Deployment

The Robot Operating System (ROS) provides a standardised publish-subscribe communication model and a large collection of reusable tools for robotics development [18]. ROS Noetic, the final ROS 1 release targeting Ubuntu 20.04, is the version most broadly compatible with the Franka's official software stack. Docker containers provide a standard method for packaging the entire software stack, enabling reproducible deployments across different host machines without dependency conflicts. Using Docker with hardware-interfacing applications introduces specific challenges that are documented in Section 3.

3 Hardware and Software Stack

This section describes all hardware components and software layers that constitute the system. The full physical setup is shown in Figure 1.

3.1 The Franka FR3 Robot Arm

The manipulation platform is a **Franka FR3**, a seven-degree-of-freedom collaborative robot arm designed for research and light industrial use. The FR3 is the successor to the Franka Emika Panda and shares the same fundamental kinematic structure: seven revolute joints with integrated torque sensors, enabling compliant control modes and collision detection. The arm has a maximum reach of approximately 855 mm and a payload capacity of 3 kg. All joints are driven by brushless DC motors with integrated encoders, and joint states are available to the controller at 1 kHz.

The arm communicates with the host workstation over Ethernet via the **Franka Control Interface (FCI)**, a proprietary low-latency protocol that operates at 1 kHz and requires deterministic communication timing. Figure 3 shows the complete system architecture from the official Franka documentation, illustrating the connection path from the workstation through the FCI control box to the arm and the External Activation Device (EAD).

The FCI is activated through the Franka Desk web interface (accessible at the robot's IP address `147.250.35.11`); a browser popup must remain open for the duration of any FCI session. The robot's joint limits are summarised in Table 1. Joint 1 (the base rotation) spans ± 2.897 rad (± 166 degrees), which provides sufficient range for the 165-degree rotation required by the placement task.

Table 1: Franka FR3 joint limits.

Joint	Min (rad)	Max (rad)	Range (deg)
J1 (base)	-2.897	$+2.897$	332
J2 (shoulder)	-1.763	$+1.763$	202
J3 (upper arm)	-2.897	$+2.897$	332
J4 (elbow)	-3.072	-0.069	173
J5 (forearm)	-2.897	$+2.897$	332
J6 (wrist 1)	-0.018	$+3.753$	215
J7 (wrist 2)	-2.897	$+2.897$	332

Figure 1: Full laboratory setup. The Franka FR3 arm is mounted on the table (left). The aluminium profile frame structure extends to the right, with the Intel RealSense D435 camera clamped to its horizontal cross-beam and aimed downward toward the workspace. A metal food can is placed on the table surface within the camera's field of view.

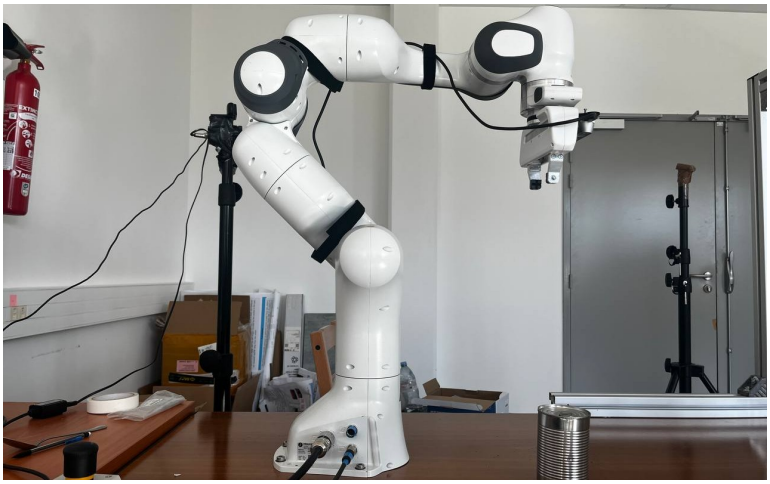


Figure 2: The Franka FR3 arm in the laboratory in its home/ready configuration. The arm's characteristic seven-joint profile is visible. The blue LED ring at the base indicates the arm is active and the brakes are released. The aluminium frame and camera stand are visible in the background.

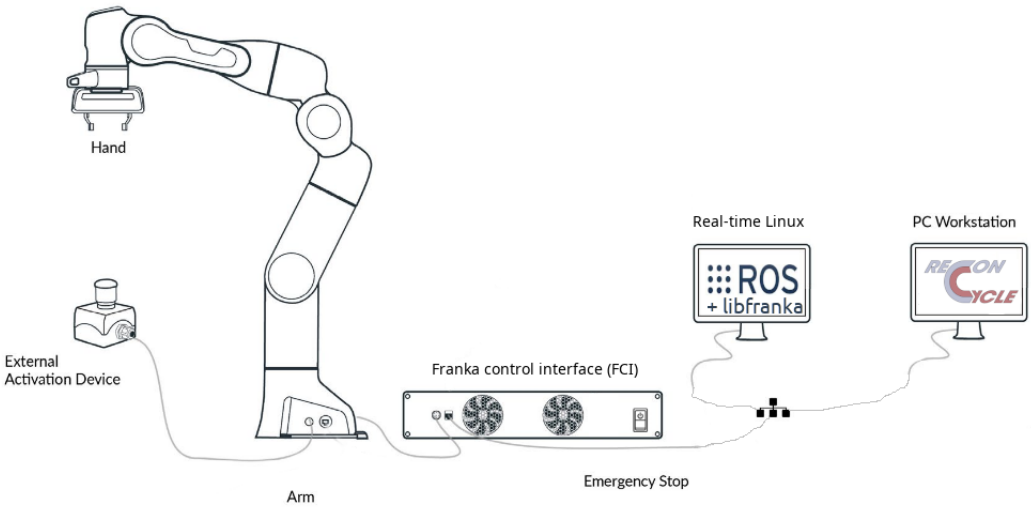


Figure 3: Official Franka system architecture diagram. The PC workstation running ROS and `libfranka` communicates with the arm through the Franka Control Interface (FCI) box via Ethernet. The External Activation Device (EAD) must remain pressed to keep the brakes released during operation. The Emergency Stop provides a hardware safety cutout.

3.2 The Franka Hand Gripper

The standard end-effector is the **Franka Hand**, a parallel-jaw gripper with a maximum opening of 8 cm. The gripper is controlled via a dedicated ROS action server at the namespace `/franka_gripper_1/franka_gripper/`, exposing `MoveAction` (for open) and `GraspAction` (for close) interfaces from the `franka_gripper` package. The grasp parameters used for metal cans are summarised in Table 2.

The large epsilon tolerance (±5 cm) is essential: it instructs the gripper to declare a grasp successful as long as the final jaw separation falls anywhere within 5 cm of the 7 cm

target. Without this tolerance the action would report failure and potentially open the gripper whenever the can was slightly off-centre, even when physically gripped.

Table 2: Gripper parameters used for metal can manipulation.

Parameter	Value	Notes
Open width	0.08 m	Full opening before approach
Grasp target width	0.07 m	Stops at first contact with can
Closing speed	0.01 m/s	Slow closure for gentle contact detection
Grasp force	5 N	Holds can without deforming it
Epsilon inner	0.05 m	Tolerance band (inner)
Epsilon outer	0.05 m	Tolerance band (outer)

3.3 Intel RealSense D435 Depth Camera

The vision system is built around an **Intel RealSense D435** RGB-D camera. The D435 provides aligned colour and depth streams at up to 30 fps, with factory-calibrated intrinsic parameters accessible via the `/camera/color/camera_info` ROS topic.

The camera is mounted on a custom **aluminium profile frame** that spans the width of the table and is fixed to the table surface with clamps. It is clamped to the horizontal cross-beam of this frame, positioned above the far end of the workspace, with a forward tilt of approximately 30 degrees. This geometry provides a clear top-down view of the entire pickup area while keeping the robot arm and its cables visible only at the edges of the frame. The camera topics used in this project are:

- `/camera/color/image_raw` — RGB colour stream at 15 fps.
- `/camera/aligned_depth_to_color/image_raw` — depth stream aligned pixel-for-pixel to the colour frame at 15 fps.
- `/camera/color/camera_info` — factory intrinsic calibration (f_x, f_y, c_x, c_y), read once at startup.

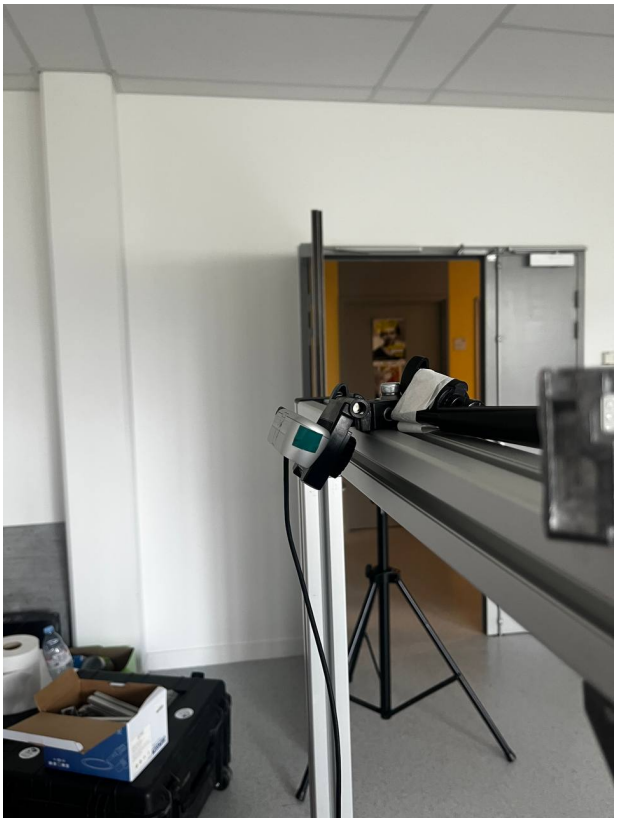
3.4 Software Stack Overview

The host machine runs **Ubuntu 22.04** with **ROS 2 Humble**. All robotics software, however, runs inside a **Docker container** based on the image `u2is-franka-panda-frida-21-11`, which encapsulates **ROS Noetic** (ROS 1), the `franka-interface` C++ backend, the `ros_interface` ROS nodes, the RealSense driver, and all Python dependencies. Table 3 summarises the principal components and where they run.

3.4.1 FrankaPy

FrankaPy [17] is a Python library providing a high-level API for controlling the FR3. It wraps the `franka-interface` action server and exposes primitives including `fa.goto_pose()`, `fa.goto_joints()`, and `fa.reset_joints()`. All motion calls use `use_impedance=False` and `ignore_virtual_walls=True`.

(a) Intel RealSense D435 close-up, showing the green status LED and its mounting clamp on the aluminium frame cross-beam.



(b) Camera mounting position: the D435 is clamped to the horizontal cross-beam of the aluminium profile frame, aimed downward over the workspace at approximately 30 degrees of forward tilt.

Figure 4: Intel RealSense D435 RGB-D camera and its mounting configuration on the custom aluminium frame structure.

Table 3: Software stack components and their execution context.

Component	Execution context	Notes
`franka-interface` C++ backend	Docker	Real-time FCI communication
`franka_ros_interface` nodes	Docker	ROS action servers
`franka_gripper` nodes	Docker	Gripper action server
`realsense2_camera`	Docker	Depth + colour streams
`object_detector.py`	Docker	YOLO + Hough detection
`franka_picker.py`	Docker	Arm motion + gripper
`rqt_image_view` (optional)	Docker	Visual debug output
`session_monitor.py` (optional)	Docker	Node for updates

The absence of a speed scaling parameter is a key non-obvious constraint: motion speed is controlled exclusively via the `duration` argument. All Cartesian moves use a 6-second duration and all joint-space moves use 8 seconds, values validated empirically to avoid joint velocity limit violations while keeping cycle times acceptable.

3.4.2 Docker Container Configuration

The container is launched with the following flags to enable hardware access and real-time scheduling:

```
docker run -it --rm \
  --gpus all \
  --name franka_frida-18-11-v2 \
  --net=host \
  --ipc=host \
  --privileged \
  -v /dev/bus/usb:/dev/bus/usb \
  --ulimit rtprio=99 \
  --ulimit memlock=-1 \
  --cap-add=sys_nice \
  --security-opt seccomp=unconfined \
  -e DISPLAY=$DISPLAY \
  -e QT_X11_NO_MITSHM=1 \
  -e XDG_RUNTIME_DIR=/tmp \
  -v /tmp/.X11-unix:/tmp/.X11-unix:rw \
  -v ~/franka_ws:/ws \
  -w /ws \
  u2is-franka-panda-frida-21-11 \
  bash
```

Listing 1: Docker run command used to launch the full system.

The `-rm` flag means the container is deleted when stopped; all project scripts are stored in the bind-mounted host directory `~/franka_ws` and cloned from the GitHub repository `Anna-Sobchuk/franka` at `~/franka` inside the container. This means code changes made on the host are immediately visible inside the container without any rebuild.

3.4.3 Real-Time Kernel Requirement

The `libfranka` library requires the host kernel to support real-time scheduling. The laboratory workstation uses a lowlatency kernel configuration, which proved sufficient for the slow deliberate motions used in this application. The required process limits are set via the container's `ulimit` flags:

```
--ulimit rtprio=99      # real-time scheduling priority limit
--ulimit memlock=-1     # unlimited memory lock (for real-time buffers)
--cap-add=sys_nice      # allow setting elevated process priority
```

Listing 2: Real-time process limits configured via Docker flags.

3.4.4 Python Dependencies

Python dependencies are installed inside the container using:

```
pip install ultralytics --ignore-installed psutil
sudo apt-get install -y ros-noetic-realsense2-camera \
                        ros-noetic-realsense2-description
```

Listing 3: Python package installation inside the Docker container.

The principal packages used are: `ultralytics` (YOLOv8), `frankapy` (arm control), `pyrealsense2` (depth SDK), `opencv-python` (image processing), `numpy`, and `autolab_core` (rigid-body transforms).

4 Object Detection Pipeline

The object detection pipeline is responsible for identifying the metal can in the camera's field of view, estimating its 3D position in camera coordinates, and delivering a stable position estimate to the robot controller. The challenge in this application is significant: the camera observes a real laboratory workspace containing numerous unrelated objects yet must precisely identify and localise the can among them. This section describes the dual-method detection approach developed to meet that challenge.

4.1 Top-Down Workspace View

Figure 5 shows the camera's perspective of the workspace. The top-down view captures the robot base, the table surface, and the can, but also reveals the degree of visual clutter present in the environment and the difficulty of singling out the metallic can from the surrounding scene.

4.2 Dual Detection Strategy: YOLO and HoughCircles

The pipeline implements two detection methods that run in parallel on every colour frame:

1. **YOLOv8n** — a neural network trained on COCO, which detects objects by appearance. This method is fast and context-aware but is limited by the absence of a dedicated "can" class in COCO.

2. **HoughCircles** — a classical computer vision method that detects circular shapes by edge analysis, independent of learned appearance. This serves as a fallback when YOLO misses the metallic can.

Both methods feed a shared candidate list. If YOLO produces any detections, they are used; otherwise the HoughCircles candidates are used. The closest detection by depth is selected as the target. This design ensures the system degrades gracefully: in good conditions YOLO provides reliable category-aware detections, while in difficult lighting or with highly reflective surfaces the Hough fallback maintains coverage.

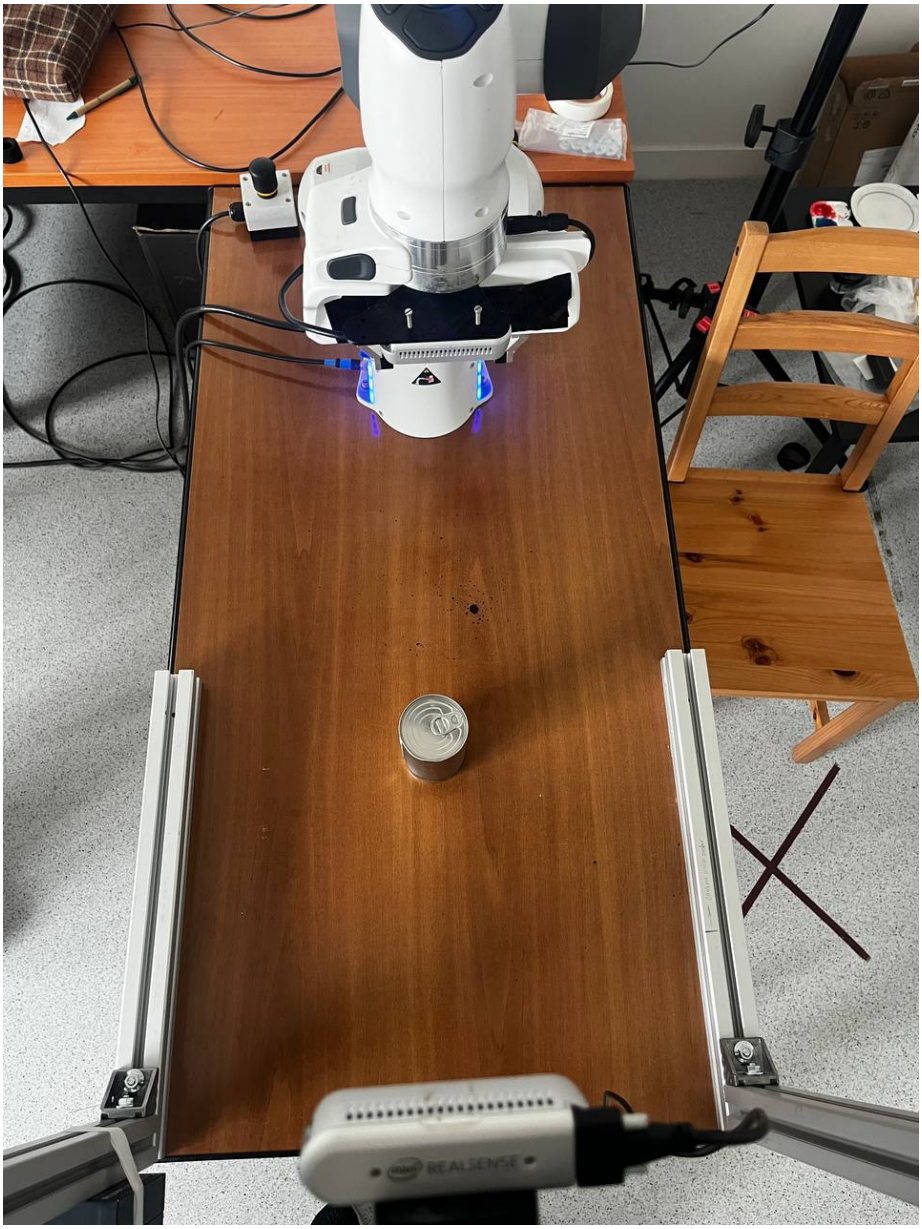


Figure 5: Top-down view of the workspace from the RealSense camera's perspective. The robot base is visible at the top of the frame; the metal food can is placed on the wooden table surface. The RealSense camera is visible at the bottom of the image in its frame-mounted position. The tape markers on the table are calibration reference positions used during the coordinate mapping.

4.3 YOLOv8 Integration

The detection model is **YOLOv8n** from the Ultralytics library, loaded with COCO-pretrained weights. The key detection parameters are:

```python
# Detection parameters
YOLO_CONF     = 0.10
YOLO_CLASSES = [39, 40, 41, 44]  # bottle, wine glass, cup, vase

MIN_DIAM = 0.05   # metres     minimum projected diameter to accept
MAX_DIAM = 0.13   # metres     maximum projected diameter to accept

def detect_yolo(self, frame):
    results = self.model(frame, verbose=False,
                         conf=YOLO_CONF, classes=YOLO_CLASSES)
    detections = []
    for r in results:
        for box in r.boxes:
            x1, y1, x2, y2 = box.xyxy[0].cpu().numpy()
            px = int((x1 + x2) / 2)
            py = int((y1 + y2) / 2)
            Z = get_depth(self.depth_image, px, py)
            if Z is None or not (MIN_DEPTH < Z < MAX_DEPTH):
                continue
            # Geometric size filter: estimate projected diameter
            est_diam = ((x2 - x1) * Z) / self.fx
            if not (MIN_DIAM < est_diam < MAX_DIAM):
                continue
            X = (px - self.cx) * Z / self.fx
            Y = (py - self.cy) * Z / self.fy
            cls_name = self.model.names[int(box.cls[0])]
            detections.append(
                (X, Y, Z, px, py, est_diam, f"YOLO:{cls_name}"))
    return detections
```

Listing 4: YOLOv8 detection call with size-based filtering in `object_detector.py`.

4.3.1 Confidence Threshold and Class Selection

The COCO dataset provides no dedicated "metal can" class. The four closest proxy classes `bottle` (39), `wine glass` (40), `cup` (41), and `vase` (44) are used as candidates. The confidence threshold is set to **0.10**: high enough to reduce noise but low enough to catch the can in typical laboratory lighting. A threshold of 0.30 was found to produce too many missed detections when the can was stationary under fluorescent light.

4.3.2 Geometric Diameter Filter

Even at a confidence of 0.10, the YOLO detector may fire on other objects in the scene. A second gate rejects any detection whose projected metric diameter falls outside the range $[0.05, 0.13]$ m:

$$d_{\text{proj}} = (x_2 - x_1) \cdot \frac{Z}{f_x} \tag{1}$$

where $(x_2 - x_1)$ is the bounding-box width in pixels, Z is the depth at the box centre, and f_x is the horizontal focal length. A standard food can (diameter ≈ 7.5 cm) falls comfortably within

this range at camera-to-table distances of 0.60–0.80 m. This filter eliminates the great majority of false positives that arise from other cylindrical objects in the laboratory environment.

4.4 HoughCircles Fallback

When YOLO produces no detections, the HoughCircles fallback is used. Before circle detection, CLAHE contrast enhancement is applied to the greyscale image to improve edge visibility on the low-contrast metallic surface, followed by Gaussian blurring to suppress fine texture noise:

```
HOUGH_MIN_RADIUS = 18
HOUGH_MAX_RADIUS = 80
HOUGH_PARAM1     = 60
HOUGH_PARAM2     = 28

def detect_hough(self, frame):
    gray  = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    # CLAHE: improves contrast on metallic/reflective surfaces
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    gray  = clahe.apply(gray)
    gray  = cv2.GaussianBlur(gray, (9, 9), 2)

    circles = cv2.HoughCircles(
        gray, cv2.HOUGH_GRADIENT, dp=1.2, minDist=60,
        param1=HOUGH_PARAM1, param2=HOUGH_PARAM2,
        minRadius=HOUGH_MIN_RADIUS,
        maxRadius=HOUGH_MAX_RADIUS
    )
    detections = []
    if circles is None:
        return detections
    for c in circles[0]:
        px, py, radius = int(c[0]), int(c[1]), int(c[2])
        Z = get_depth(self.depth_image, px, py)
        if Z is None or not (MIN_DEPTH < Z < MAX_DEPTH):
            continue
        est_diam = (radius * 2 * Z) / self.fx
        if not (MIN_DIAM < est_diam < MAX_DIAM):
            continue
        X = (px - self.cx) * Z / self.fx
        Y = (py - self.cy) * Z / self.fy
        detections.append(
            (X, Y, Z, px, py, est_diam, "Hough"))
    return detections
```

Listing 5: HoughCircles fallback detector with CLAHE preprocessing in `object_detector.py`.

CLAHE (Contrast Limited Adaptive Histogram Equalisation) enhances local contrast by operating on small tiles of the image rather than the whole frame. On a metallic can, this recovers edge detail that is washed out by specular highlights in the standard greyscale image, significantly improving circle detection reliability. As shown in the terminal output from

multiple consecutive runs (Section 9.3), the HoughCircles detector was in practice the primary detection source for the metallic food cans used in this project.

4.5 Depth Back-Projection

For each candidate detection, the 3D position in the camera optical frame is computed from the 2D pixel location (u, v) using the depth Z and the camera intrinsics (f_x, f_y, c_x, c_y):

$$X_c = \frac{(u - c_x) \cdot Z}{f_x}, \quad Y_c = \frac{(v - c_y) \cdot Z}{f_y}, \quad Z_c = Z \tag{2}$$

The depth value Z is obtained from the aligned depth frame using a **robust 11×11 pixel median** to handle invalid depth readings caused by reflective surfaces:

```python
def get_depth(depth_image, px, py, window=11):
    """
    Robust depth: median of valid (non-zero) pixels in an 11x11 window.
    Handles missing depth caused by specular reflection on metal cans.
    Returns depth in metres, or None if fewer than 5 valid readings.
    """
    h, w = depth_image.shape
    half = window // 2
    roi = depth_image[
        max(0, py - half):min(h, py + half + 1),
        max(0, px - half):min(w, px + half + 1)
    ]
    valid = roi[roi > 0]
    if valid.size < 5:
        return None
    return np.median(valid) / 1000.0    # uint16 mm -> float metres
```

Listing 6: Robust depth sampling in `object_detector.py`. A large window and 20th-percentile value handle the missing depth pixels common on metallic surfaces.

The 11×11 window is deliberately large to sample pixels away from the can's specular centre, where depth returns are most often invalid. The `valid.size < 5` guard rejects the detection entirely if fewer than five valid depth readings are found, preventing the computation of nonsensical 3D positions from highly reflective surfaces.

4.6 Stability Buffer

A single-frame detection is insufficient to trigger robot motion. A **stability buffer** requiring **8 consecutive frames** within **0.025 m** of each other guards against noise, transient occlusions, and spurious detections from passing hands or moving objects:

```
STABILITY_FRAMES    = 8
STABILITY_MAX_DRIFT = 0.025  # metres

# Per-frame stability update (inside color_cb):
if self.armed and best is not None:
    X, Y, Z, px, py, diam, label = best
    pos = (X, Y, Z)
    if self.last_pos is None:
        self.stable_count = 1
        self.last_pos     = pos
    else:
        drift = np.linalg.norm(np.array(pos) - np.array(self.last_pos))
        if drift < STABILITY_MAX_DRIFT:
            self.stable_count += 1
            self.last_pos      = pos
        else:
            # Can moved or different object detected     reset count
            self.stable_count = 1
            self.last_pos      = pos

    if self.stable_count >= STABILITY_FRAMES:
        self.armed        = False   # disarm     robot will now move
        self.stable_count = 0
        self.last_pos     = None
        # Publish PointStamped to picker
        pt = PointStamped()
        pt.header.stamp    = rospy.Time.now()
        pt.header.frame_id = 'camera_color_optical_frame'
        pt.point.x, pt.point.y, pt.point.z = X, Y, Z
        self.point_pub.publish(pt)
elif best is None:
    # No detection this frame     reset stability count
    self.stable_count = 0
    self.last_pos     = None
```

Listing 7: Stability logic in `object_detector.py`. The can must remain stationary for 8 consecutive frames within a 2.5 cm drift tolerance before the picker is triggered.

At 15 fps, eight frames correspond to approximately half a second. The visual feedback of stability progress is provided in the debug image stream via a real-time progress bar overlaid on the camera frame, and the current detection method (YOLO or Hough) is shown in the status overlay.

4.7 Detection in a Cluttered Environment

One of the most practically significant results of this project is that the dual detection system reliably identifies the target can in a genuinely cluttered laboratory scene. Figure 6 shows the debug view from the `/object_detector/yolo_debug` topic during a live run.

Figure 6: Debug detection output (`/object_detector/yolo_debug` topic). The workspace is viewed from above. The metal can is detected and highlighted with a bounding box; the estimated diameter and depth ($D = 7.8$ cm, $Z = 0.70$ m) are shown alongside the detection method label (YOLO:`cup` and Hough both active). The red dot marks the sampled 3D back-projection point. The "ROBOT BUSY" overlay in the top-left corner indicates the picker is executing a motion and the detector is in its disarmed state. Despite the visible clutter in the environment — including the robot base itself and other objects — the system reliably focuses on the can.

The scene in Figure 6 contains not only the can but also the robot base, cables, the aluminium frame rails, and various background items. The diameter filter and the depth range constraints together suppress the vast majority of irrelevant regions. Neither the robot body, the frame rails, nor any background items were observed to trigger a false positive in normal operation, because none of them present a circular profile at the expected depth and diameter range of a 7.5 cm can at 0.60–0.80 m depth.

5 Coordinate Frame Calibration

The object detection pipeline produces positions in the camera's optical frame, but the robot's motion controller requires positions in the robot base frame (`panda_link0`). This section describes the two calibration approaches attempted, explains why the geometric TF approach was abandoned, and derives the empirical regression mapping that was ultimately used.

5.1 The Camera-to-Robot Transform Problem

The camera is mounted statically on the aluminium frame above the table. The required transform is therefore a rigid-body *eye-to-base* transform ${}^{B}T_{C}$ that maps a point in camera frame C to the robot base frame B:

$$\begin{pmatrix} X_B \\ Y_B \\ Z_B \\ 1 \end{pmatrix} = {}^{B}T_{C} \cdot \begin{pmatrix} X_C \\ Y_C \\ Z_C \\ 1 \end{pmatrix} \tag{3}$$

where ${}^{B}T_{C} \in SE(3)$ is a 4×4 homogeneous transform with six independent parameters (three translation, three rotation).

5.2 Approach 1: ROS TF Static Transform

The first approach published a static TF transform via a dedicated `camera_tf_publisher.py` node, declaring the camera pose relative to `panda_link0`. The physical mounting position was measured manually on 2026-03-12 using the can-touch method: the robot gripper was driven to touch the top of the can at several positions; the known gripper coordinates in robot frame were compared to the simultaneously reported camera coordinates to derive the translation and rotation parameters.

The rotation was composed from a 180-degree flip around the X-axis (required because a downward-facing optical frame has its Z-axis pointing away from the scene) combined with a 30-degree tilt around Y to account for the physical camera mounting angle:

```
# Calibrated translation (panda_link0 -> camera_color_optical_frame)
static_transform.transform.translation.x = 0.6912
static_transform.transform.translation.y = 0.2925
static_transform.transform.translation.z = 0.8286

# Rotation: 180deg flip around X + 30deg tilt forward around Y
rot_down = tft.quaternion_from_euler(math.pi, 0, 0)
rot_tilt = tft.quaternion_from_euler(0, math.radians(-30), 0)
combined_quat = tft.quaternion_multiply(rot_down, rot_tilt)
```

Listing 8: Calibrated TF transform parameters in `camera_tf_publisher.py` (legacy, calibrated 2026-03-12).

While this approach produced reasonable results near the calibration reference point, systematic position errors appeared at other positions on the table. The root cause is that angular errors in the rotation matrix are amplified by scene depth: a one-degree angular error at a camera height of 80 cm translates into a positional error of approximately 1.4 cm in the robot

frame and with multiple unknown rotation parameters, the cumulative error was large enough to cause consistent grasp misses away from the calibration point. The `camera_tf_publisher` node also had to be kept running throughout every session, and any restart briefly interrupted the TF tree and could cause lookup failures in the detector.

5.3 Approach 2: Regression-Based Mapping

The second approach treats the mapping as a direct regression problem. A set of correspondence measurements $(c_x, c_y, c_z) \to (r_x, r_y)$ was collected and a linear model was fitted.

5.3.1 Calibration Data Collection

Four calibration measurements were collected by placing the can at known positions across the workspace and recording: (i) the camera-frame 3D coordinates reported by the detector, and (ii) the robot end-effector position when the gripper physically touched the top of the can. The positions were spread across the reachable workspace to ensure the model generalises over the full pick area.

5.3.2 Linear Regression Model

A linear model was fitted mapping $(c_x, c_y, c_z, 1)$ to (r_x, r_y):

$$r_x = 0.1396\, c_x + 1.6647\, c_y + 0.3942\, c_z + 0.0982 \tag{4}$$

$$r_y = 1.0213\, c_x - 0.0325\, c_y - 0.1186\, c_z + 0.0141 \tag{5}$$

The robot-frame Z coordinate (grasp height) is not regressed but fixed at the measured mean table height `TABLE_Z = 0.0983 m`. The can-specific grasp height is then:

$$r_z = \texttt{TABLE_Z} + \texttt{CAN_HEIGHT} - 0.01 = 0.0983 + 0.11 - 0.01 = 0.1983 \text{ m} \tag{6}$$

The implementation in `franka_picker.py` is:

```
# Learned from 4 calibration points using linear regression
CAM2ROBOT_X = [0.1396,  1.6647,  0.3942,  0.0982]  # [cx, cy, cz, 1]
CAM2ROBOT_Y = [1.0213, -0.0325, -0.1186,  0.0141]  # [cx, cy, cz, 1]
TABLE_Z = 0.0983   # mean table height in robot frame (metres)

# Applied at runtime when a detection arrives:
cam_x, cam_y, cam_z = msg.point.x, msg.point.y, msg.point.z
cx = (CAM2ROBOT_X[0] * cam_x + CAM2ROBOT_X[1] * cam_y
    + CAM2ROBOT_X[2] * cam_z + CAM2ROBOT_X[3])
cy = (CAM2ROBOT_Y[0] * cam_x + CAM2ROBOT_Y[1] * cam_y
    + CAM2ROBOT_Y[2] * cam_z + CAM2ROBOT_Y[3])
cz = TABLE_Z   # vertical position fixed to table height
```

Listing 9: Direct camera-to-robot coordinate mapping in `franka_picker.py`. No TF publisher node is required.

The large coefficient on c_y in the r_x equation (1.6647) reflects the primary depth-to-forward-distance mapping: the camera's optical Y-axis (pointing downward in the scene) approximately corresponds to forward distance in the robot frame. The c_x coefficient in the r_y equation (1.0213) reflects the lateral correspondence. The remaining cross-terms capture the non-orthogonality introduced by the camera's 30-degree forward tilt.

5.4 Workspace Validity Check

Before any motion begins, the computed robot-frame position is checked against the physically reachable and safe workspace bounds:

```
# Reject positions outside the robot's reachable and safe workspace
if not (0.25 < cx < 0.80) or not (-0.40 < cy < 0.40):
    rospy.logwarn(
        f"Position out of workspace (X={cx:.3f} Y={cy:.3f}) -- skipping")
    self.busy = False
    self.done_pub.publish(Bool(data=True))    # re-arm detector
    return
```

Listing 10: Workspace reachability check in `franka_picker.py`.

This check prevents the robot from attempting to reach positions near the table edge, behind the frame structure, or at the limits of its kinematic reach where trajectory planning is unreliable.

5.5 Elimination of the TF Publisher Node

A significant practical benefit of the regression approach is the complete removal of the `camera_tf_publisher` node from the running system. The regression coefficients are six scalar constants embedded directly in `franka_picker.py`, requiring no running node, no ROS parameter server, and no TF tree. This eliminated an entire class of session startup errors and simplified the launch procedure.

5.6 Recalibration Procedure

If the camera or its mounting frame is moved, the mapping must be recalibrated. The procedure is: place the can at several known positions spanning the workspace, record the camera output from `rostopic echo /detected_object/point` and the robot end-effector position from `fa.run_guide_mode(duration=10)`, then fit new regression coefficients and update `CAM2ROBOT_X` and `CAM2ROBOT_Y` in `franka_picker.py`.

6 Robot Motion Control

This section describes the robot's physical motion: how the arm and gripper are commanded, how the pick-and-place sequence was designed and validated, and what difficulties were encountered. Two complementary scripts implement the motion: `manual_test.py` establishes a validated baseline with hardcoded coordinates, and `franka_picker.py` implements the autonomous pipeline that uses live detections.

6.1 The Manual Baseline Script

Before introducing any perception, a **manual test script** (`manual_test.py`) was developed to verify that the arm and gripper work correctly with hardcoded coordinates. This isolated the motion problem from the perception problem and produced a reference motion sequence whose parameters were validated empirically. The exact joint configurations and timing values established in this script were reused in the autonomous picker.

The motion parameters are defined as follows:

```
CAN_X        = 0.50   # metres forward from robot base
CAN_Y        = 0.00   # metres lateral (0 = centred)
CAN_HEIGHT   = 0.11   # can height (11 cm)
CAN_DIAMETER = 0.075  # can diameter (7.5 cm)

GRASP_Z = CAN_HEIGHT - 0.01   # = 0.10 m (1 cm from can top)
HOVER_Z = GRASP_Z + 0.15      # = 0.25 m (hover height)
LIFT_Z  = GRASP_Z + 0.30      # = 0.40 m (safe rotation height)
PLACE_Z = CAN_HEIGHT - 0.02   # = 0.09 m (placement height)

CART_DURATION  = 6   # seconds for Cartesian moves
JOINT_DURATION = 8   # seconds for joint-space moves
```

Listing 11: Geometry and timing constants in `manual_test.py`.

6.2 Gripper Control

6.2.1 Namespace Mismatch and the Direct Actionlib Solution

FrankaPy provides built-in gripper methods (`fa.open_gripper()`, `fa.close_gripper()`). In the laboratory environment these methods were found to hang indefinitely. The root cause was a **namespace mismatch**: FrankaPy expects the gripper action server at `/franka_gripper/`, but the server in this environment runs at `/franka_gripper_1/franka_gripper/`. FrankaPy exposes no configuration parameter to change this namespace.

The solution bypasses FrankaPy's gripper API entirely, communicating directly with the action server via ROS `actionlib`:

```
GRIPPER_NS = '/franka_gripper_1/franka_gripper'

class GripperController:
    def __init__(self):
        self.move_client  = actionlib.SimpleActionClient(
            f'{GRIPPER_NS}/move', MoveAction)
        self.grasp_client = actionlib.SimpleActionClient(
            f'{GRIPPER_NS}/grasp', GraspAction)
        self.move_client.wait_for_server(timeout=rospy.Duration(10))
        self.grasp_client.wait_for_server(timeout=rospy.Duration(10))

    def open(self, width=0.08, speed=0.05):
        goal = MoveGoal(width=width, speed=speed)
        self.move_client.send_goal(goal)
        self.move_client.wait_for_result(rospy.Duration(10))

    def close(self):
        """Close slowly with low force -- stops at first contact."""
        goal = GraspGoal()
        goal.width   = 0.07          # target width (7 cm)
        goal.speed   = 0.01          # closing speed (1 cm/s)
        goal.force   = 5.0           # max force (5 N)
        goal.epsilon = GraspEpsilon(inner=0.05, outer=0.05)
        self.grasp_client.send_goal(goal)
        self.grasp_client.wait_for_result(rospy.Duration(10))
        result = self.grasp_client.get_result()
        return result.success if result else False
```

Listing 12: `GripperController` class in `franka_picker.py`: direct `actionlib` interface at the correct namespace.

The slow closing speed (1 cm/s) and the generous epsilon tolerance (±5 cm) together produce reliable, gentle grasps: the gripper stops at first contact with the can and declares success for any final jaw separation within 5 cm of the 7 cm target, regardless of whether the can is gripped slightly off-centre.

6.3 The Two-Step Rotation Problem and Solutions

6.3.1 Initial Failure: Single-Joint Rotation

The first attempt to place the can behind the robot commanded only joint 0 (the base joint) directly from a forward-facing pick position to 165 degrees. This caused the arm to pass through a near-singular configuration midway through the rotation, producing unpredictable large joint velocities and in one case a collision with the edge of the table.

6.3.2 Manual Test Solution: Via 90-Degree Midpoint

The `manual_test.py` solution introduces a safe 90-degree sideways waypoint, splitting the rotation into two smooth joint-space moves. All seven joint angles are specified simultaneously at each waypoint rather than changing only joint 0, which eliminates the singular-path problem:

```
# All seven joint angles specified simultaneously (radians).
# Determined empirically -- collision-free for this workspace geometry.
MID_JOINTS = [
    np.radians(90),     # J1: arm faces sideways
    np.radians(-45),    # J2
    np.radians(0),      # J3
    np.radians(-135),   # J4: elbow bent for safe clearance
    np.radians(0),      # J5
    np.radians(90),     # J6: wrist
    np.radians(45),     # J7
]
BEHIND_JOINTS = [
    np.radians(165),    # J1: arm faces behind robot
    np.radians(-45),    # J2
    np.radians(0),      # J3
    np.radians(-135),   # J4
    np.radians(0),      # J5
    np.radians(90),     # J6
    np.radians(45),     # J7
]

# Step 7a: rotate to 90-degree sideways midpoint
goto_joints(fa, MID_JOINTS)

# Step 7b: continue to 165-degree behind configuration
goto_joints(fa, BEHIND_JOINTS)
```

Listing 13: Two-step rotation waypoints in `manual_test.py`: via 90-degree sideways midpoint to 165-degree behind configuration.

6.3.3 Autonomous Picker Solution: Via Safe Centred Configuration

The autonomous `franka_picker.py` uses a different rotation strategy that is more robust across variable pick positions. Rather than always passing through the 90-degree sideways waypoint, it first moves all joints to a **safe centred intermediate configuration** (J1 = 0 degrees), then rotates directly to 165 degrees. This centred intermediate is always reachable from any pick position regardless of the starting arm pose, unlike the 90-degree waypoint which was validated only for the centred pick position:

```
# Step 7a: move to safe centred configuration (joint 0 = 0 deg).
# This intermediate is always reachable from any pick position.
SAFE_LIFTED_JOINTS = [
    np.radians(0),     # J1: centred
    np.radians(-45),
    np.radians(0),
    np.radians(-135),
    np.radians(0),
    np.radians(90),
    np.radians(45),
]
goto_joints(self.fa, SAFE_LIFTED_JOINTS)

# Check can is still held after first rotation
w_after = get_gripper_width()
if w_after < 0.020:    # < 2 cm means can was dropped
    rospy.logwarn("Object dropped during rotation -- aborting")
    self.fa.reset_joints()
    self.done_pub.publish(Bool(data=True))
    return

# Step 7b: rotate directly to behind-robot configuration (165 deg)
BEHIND_JOINTS = [
    np.radians(165),
    np.radians(-45),
    np.radians(0),
    np.radians(-135),
    np.radians(0),
    np.radians(90),
    np.radians(45),
]
goto_joints(self.fa, BEHIND_JOINTS)
```

Listing 14: Two-step rotation in `franka_picker.py`: via safe centred configuration (0°) directly to behind position (165°).

The gripper width check between the two rotation steps is a critical safety feature: it detects a dropped can before the arm sweeps to the rear position, preventing the system from completing a full rotation unnecessarily and returning home so the detector can acquire a fresh view.

6.4 Gripper Tilt for Extended Reaches

For can positions at a forward distance greater than 0.60 m (X > 0.60), the wrist approaches a singularity when the gripper is held perfectly vertical. A 10-degree forward tilt is applied to the end-effector orientation for these extended positions:

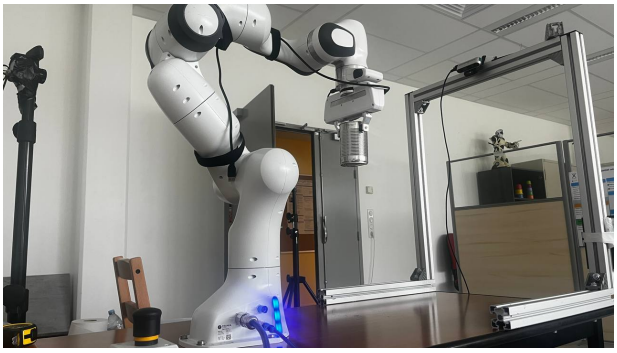
```python
def make_pose(x, y, z, tilt_deg=0):
    """Pose pointing straight down; tilt_deg tilts forward around Y."""
    if tilt_deg == 0:
        rot = ROTATION_DOWN
    else:
        angle = np.radians(tilt_deg)
        c, s = np.cos(angle), np.sin(angle)
        tilt = np.array([[c, 0, s], [0, 1, 0], [-s, 0, c]])
        rot = ROTATION_DOWN @ tilt
    return RigidTransform(rotation=rot,
                          translation=np.array([x, y, z]),
                          from_frame='franka_tool', to_frame='world')

# Applied automatically based on X position:
tilt = 10 if cx > 0.60 else 0
goto(self.fa, make_pose(cx, cy, hover_z, tilt_deg=tilt))
```

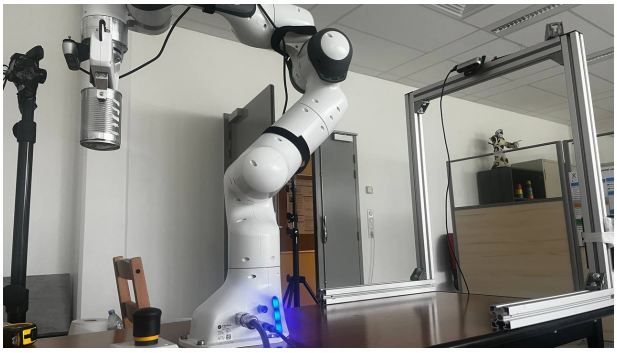
Listing 15: Automatic tilt for extended reaches in `franka_picker.py`.

6.5 The Full Pick-and-Place Sequence

The complete 11-step autonomous pick-and-place sequence implemented in `franka_picker.py` is illustrated by Figure 7 and described in Table 4. The descent to grasp height is split into two sub-steps (4a and 4b) to prevent the franka-interface from silently rejecting a large single downward move.



(a) The gripper closed on the can and the arm lifting to the safe rotation height ($z = 0.40$ m). The can is held firmly in the Franka Hand parallel-jaw gripper.

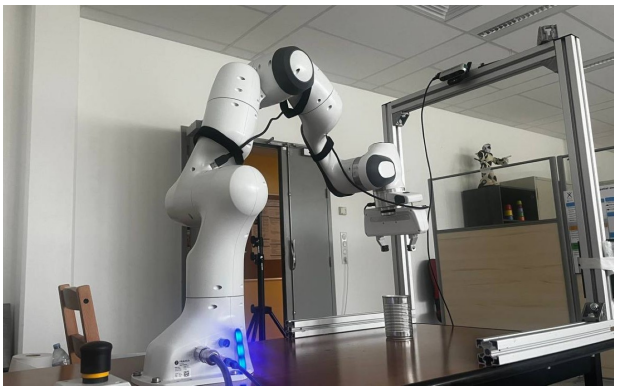


(b) The arm after the 165-degree rotation, with the can in the gripper. The arm now faces behind the robot. The can will be lowered to the table at this position before being released.

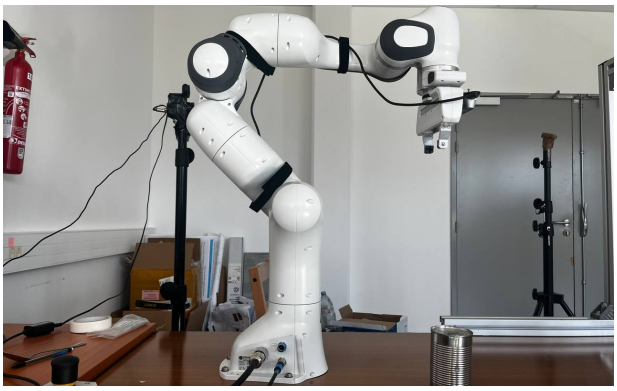
Figure 7: Two key stages of the pick-and-place sequence. Left: can securely gripped and lifted. Right: arm rotated 165° and can positioned above the placement point behind the robot.

Table 4: Pick-and-place sequence in `franka_picker.py`.

Step	Action	Type	Duration
1	Home (reset joints)	Joint reset	—
2	Open gripper to 8 cm	Gripper move	—
3	Hover above can (z=0.25 m)	Cartesian	6 s
4a	Lower to mid-height	Cartesian	6 s
4b	Lower to grasp height (z=0.10 m)	Cartesian	6 s
Verify: abort if actual $z >$ target + 5 cm			
5	Close gripper (5 N, 7 cm target)	Gripper grasp	—
Verify: abort if gripper width $<$ 2 cm			
6	Lift to $z = 0.40$ m	Cartesian	6 s
7a	Rotate to safe centred config (0°)	Joint-space	8 s
Verify: abort if gripper dropped can			
7b	Rotate to behind-robot (165°)	Joint-space	8 s
8	Lower can to table ($z = 0.09$ m)	Cartesian	6 s
9	Open gripper (slow release)	Gripper move	—
10	Lift away from placed can	Cartesian	6 s
11	Return home	Joint reset	—



(a) The arm in a mid-motion configuration after the rotation. The can is visible on the right side of the table, having been released at the placement position behind the robot. The arm is lifting away after successful placement.



(b) The Franka FR3 arm in its home/reset configuration, ready for the next detection cycle. The arm returns to this position at the end of every successful or failed pick attempt.

Figure 8: Completion of the pick-and-place cycle. Left: arm lifting away after placing the can. Right: arm returned to the home configuration, detector re-armed.

7 System Integration

This section explains how the subsystems are connected into a coherent, autonomous pipeline: the ROS node architecture, the design of the inter-node communication protocol, the arm/disarm synchronisation mechanism, the fault-tolerance strategy, and the containerisation approach that ensures reproducible deployment.

7.1 ROS Node Architecture

The system consists of two principal application nodes communicating through a minimal set of ROS topics:

`object_detector.py` Subscribes to the RealSense colour and depth streams, runs the dual YOLO/Hough detection pipeline, applies the stability buffer, and publishes a single `geometry_msgs/PointStamped` on `/detected_object/point` when a stable detection is confirmed. After publishing, the node *disarms* itself and ignores all subsequent frames until the picker signals completion.

`franka_picker.py` Subscribes to `/detected_object/point`. On receipt, it applies the regression coordinate mapping, validates workspace bounds, and executes the full pick-and-place sequence. On completion (successful or not), it publishes `True` on `/franka_picker/done`, which re-arms the detector for the next cycle.

The complete set of ROS topics used by the system is listed in Table 5.

Table 5: ROS topics used by the system.

Topic	Message type	Description
`/detected_object/point`	`PointStamped`	Can position in camera frame
`/detected_can/diameter`	`Float32`	Estimated can diameter (m)
`/detected_can/height`	`Float32`	Can height (fixed 0.11 m)
`/franka_picker/done`	`Bool`	Picker done signal (re-arms detector)
`/object_detector/yolo_debug`	`Image`	Annotated debug camera feed
`/camera/color/image_raw`	`Image`	Raw RealSense colour stream
`/camera/aligned_depth_to_color/image_raw`	`Image`	Aligned depth stream

7.2 Inter-Node Communication Protocol Design

ROS provides three principal inter-process communication mechanisms: topics (asynchronous publish-subscribe), services (synchronous request-response), and actions (asynchronous request with ongoing feedback and preemption). The choice between them has direct consequences for system reliability and coupling.

A ROS *service* call from the picker to the detector to signal completion would appear superficially natural, but introduces a blocking dependency: the picker would have to wait for the detector's service handler to respond before proceeding to its home step. If the detector has crashed or is slow, the picker would block indefinitely. More fundamentally, services are synchronous by design and would introduce tight temporal coupling between two nodes that are otherwise entirely independent in their execution rate.

A ROS *action* would be appropriate if the picker needed to provide ongoing feedback, heartbeats, partial results, or preemption support, to the detector. None of these are required: the detector needs to know only whether the robot is busy or free, a binary state. The action framework's overhead of separate goal, result, and feedback message types, plus state machines in both client and server, is unjustified for a signal whose entire content is a single boolean.

The *topic* approach adopted here reflects a deliberate design choice for minimal coupling. Two asymmetric topics carry the complete handshake:

- `/detected_object/point` carries the 3D can position from detector to picker. Its queue size is set to 1. If the picker is busy and misses a detection message, it is better to discard it than to queue it, because a stale position becomes meaningless within seconds as the robot moves and the scene changes.

- `/franka_picker/done` carries `Bool(True)` from picker to detector. Its queue size is also 1. Re-arm signals do not accumulate meaningfully, and double re-arming is harmless because the detector checks the `armed` flag before acting on any detection.

The result is a fully decoupled architecture in which neither node ever blocks waiting for the other. The detector runs its detection loop at the camera frame rate regardless of what the picker is doing, and the picker executes its motion sequence regardless of the detector's state.

7.3 Arm/Disarm Synchronisation

The arm/disarm mechanism is the key to multi-cycle autonomous operation. Without it, the detector could publish a new target while the robot is mid-motion, causing conflicting goals or missed messages.

In `object_detector.py`, the `armed` flag is set to `False` immediately after publishing a detection and only reset to `True` when `True` is received on `/franka_picker/done`:

```python
def done_cb(self, msg):
    """Re-arm when picker signals completion."""
    if msg.data:
        self.armed        = True
        self.stable_count = 0
        self.last_pos     = None
        rospy.loginfo("Detector re-armed.")
```

Listing 16: Arm/disarm logic in `object_detector.py`.

In `franka_picker.py`, the `finally` block ensures the detector is always re-armed, even if the motion sequence raises an exception:

```
try:
    # ... full pick-and-place sequence ...
except Exception as e:
    rospy.logerr(f"Pick & place failed: {e}")
    try:
        self.fa.reset_joints()    # always try to return home on error
    except:
        pass
finally:
    self.busy = False
    rospy.sleep(2.0)    # wait for arm to clear camera view
    self.done_pub.publish(Bool(data=True))    # re-arm detector
```

Listing 17: The `finally` block in `franka_picker.py` guarantees the done signal is always published, enabling uninterrupted multi-cycle operation regardless of how the pick sequence terminates.

The two-second sleep before re-arming ensures the arm has moved clear of the camera's field of view, preventing the detector from immediately triggering on the arm itself after a cycle completes.

7.4 Fault Tolerance and Exception Safety

A pick-and-place system operating in a real laboratory must handle failures gracefully at multiple levels. The system implements a three-level defence-in-depth fault-tolerance strategy.

Level 1 — Pre-motion validation. Before any joint command is issued, the converted robot-frame position (r_x, r_y) is checked against the physical workspace boundary:

$$0.25 < r_x < 0.80 \;\text{ and }\; |r_y| < 0.40 \tag{7}$$

Positions outside these bounds are rejected immediately, the detector is re-armed, and the system waits for a fresh detection. This guards against coordinate mapping errors near the workspace edge, where the linear regression model extrapolates beyond its training data, and prevents the arm from attempting trajectories that exceed its kinematic reach.

Level 2 — Mid-sequence verification. Two sensor-based checkpoints are embedded within the pick sequence. After the descent to grasp height (step 4b), the actual end-effector z-coordinate is read back from the arm and compared to the commanded target:

```
actual_z = self.fa.get_pose().translation[2]
if actual_z > grasp_z + 0.05:
    rospy.logwarn(
        f"Robot did not lower to target "
        f"(at Z={actual_z:.3f}, target {grasp_z:.3f}) -- aborting")
    self.gripper.open(width=0.08)
    self.fa.reset_joints()
    rospy.sleep(2.0)
    self.busy = False
    self.done_pub.publish(Bool(data=True))
    return
```

Listing 18: Post-descent height verification. If the arm did not reach the commanded height — a symptom of a trajectory rejection by the `franka-interface` — the sequence aborts and re-arms the detector without attempting a grasp.

After the grasp (step 5), the gripper jaw width is read back from the `joint_states` topic and compared against a contact threshold:

```
gripper_w = get_gripper_width()
if gripper_w < 0.020:   # < 2 cm: jaws closed without contacting can
    rospy.logwarn("Gripper closed to 0 -- missed can. Aborting.")
    self.gripper.open(width=0.08)
    self.fa.reset_joints()
    rospy.sleep(2.0)
    self.busy = False
    self.done_pub.publish(Bool(data=True))
    return
```

Listing 19: Gripper contact verification. A width below 2 cm indicates the jaws closed through empty air (missed grasp). The sequence aborts before any rotation, returning home so the detector can acquire a fresh view.

The 2 cm threshold was chosen empirically: a successful grasp on a 7.5 cm diameter can produces a jaw separation of 3–6 cm; a missed grasp produces a separation close to zero. The 2 cm threshold provides a safe margin between these two cases while tolerating off-centre grasps at the can's narrower profile regions. A third gripper-width check occurs between the two rotation steps (between steps 7a and 7b), detecting any drop during the first joint-space move before the arm sweeps behind the robot.

Level 3 — Exception-guaranteed recovery. The `finally` block in Listing 17 ensures that `/franka_picker/done` is published regardless of how the sequence terminates — normal completion, an explicit `return` from a Level 2 check, or an unhandled exception from any FrankaPy or ROS call. Without this guarantee, a single unexpected exception would permanently disarm the detector with no visible error to the operator, leaving the system appearing healthy but silently inactive.

7.5 Containerisation and Reproducible Deployment

The entire software stack runs inside a Docker container based on the image `u2is-franka-panda-frida-21-11`. Containerisation is not merely a convenience here; it addresses three specific engineering requirements of this system.

Dependency isolation. The system requires a precise combination of ROS Noetic, the `franka-interface` C++ backend, and Python packages including `frankapy` and `ultralytics`, all of which conflict with the host machine's Ubuntu 22.04 and ROS 2 Humble environment. Docker encapsulates this dependency graph in a self-contained image, leaving the host environment unmodified and enabling the same image to be deployed on any machine with Docker and an NVIDIA GPU without any additional configuration.

Real-time process privileges. The `libfranka` library requires the ability to set real-time thread scheduling priorities (`SCHED_FIFO`), which is a privileged kernel operation unavailable to standard unprivileged processes. The container is launched with `-ulimit rtprio=99`, `-ulimit memlock=-1`, and `-cap-add=sys_nice`, granting the containerised processes the specific elevated capabilities they require without granting unrestricted root access on the host system.

GPU and hardware passthrough. YOLOv8 inference requires GPU access to achieve frame rates compatible with the 15 fps camera stream. The `-gpus all` flag and `-v /dev/bus/usb:/dev/bus/usb` device mapping allow the container to access the NVIDIA GPU for inference and the RealSense depth camera via USB, with performance equivalent to a native installation. The `-net=host` flag is required for the container to reach the robot over Ethernet at its fixed IP address.

The development workflow exploits the bind-mounted workspace volume. Project scripts are stored on the host at `~/franka_ws` and mounted into the container at `/ws`, so any edit on the host is immediately visible inside the container without a rebuild or restart. The code is additionally version-controlled at `github.com/Anna-Sobchuk/franka`, enabling the exact software state used in any given experiment to be recovered from the commit history.

8 Program Control Flow

The preceding sections described each subsystem, perception, coordinate mapping, motion control, and system integration, in analytical terms. This section consolidates that description into three annotated flowcharts that together capture the complete runtime behaviour of the system. Each chart is accompanied by the key code extracts that implement the decisions and actions shown.

The three charts cover distinct levels of the program structure: Figure 9 shows the static data-flow topology between all ROS nodes and hardware components; Figure 10 shows the per-frame control flow of the detection pipeline; and Figure 11 shows the full pick-and-place sequence with every fault-tolerance branch.

8.1 System Data-Flow Topology

Figure 9 shows the complete ROS node and hardware topology of the system. Unlike the following two charts, which describe sequential algorithms, this diagram depicts the *concurrent* data flow between processes: each node runs in its own process, and communication happens exclusively through ROS topics.

The two most important data flows are the forward detection path and the backward re-arm path. The forward path carries the can position from camera to arm: the camera publishes raw images; the detector processes them, and on stable confirmation publishes a single `PointStamped` to `/detected_object/point`; the picker receives this message, converts the coordinates, and executes the motion sequence. The backward path carries the completion signal from arm to camera: at the end of every cycle, regardless of outcome, the picker publishes `Bool(True)` to `/franka_picker/done`, which causes the detector's `done_cb` to set its `armed` flag back to `True`:

```python
def done_cb(self, msg):
    if msg.data:
        self.armed        = True
        self.stable_count = 0
        self.last_pos     = None
        rospy.loginfo("Detector re-armed.")
```

Listing 20: The `done_cb` callback in `object_detector.py`. On receipt of `True` from the picker, the detector resets its stability state and returns to active scanning.

This single callback is the complete implementation of the arm/disarm mechanism that enables uninterrupted multi-cycle operation. Without it, a new detection could be published while the arm is mid-motion, leading to conflicting goal messages or stale position data being acted upon.

8.2 Detection Pipeline

Figure 10 shows the per-frame control flow executed inside `color_cb()` on every image received from the RealSense camera at 15 fps. The flowchart makes the priority relationship between the two detection methods explicit: YOLO and HoughCircles run in parallel on every frame, but YOLO results take precedence, HoughCircles candidates are used only if YOLO produces no output after size filtering.

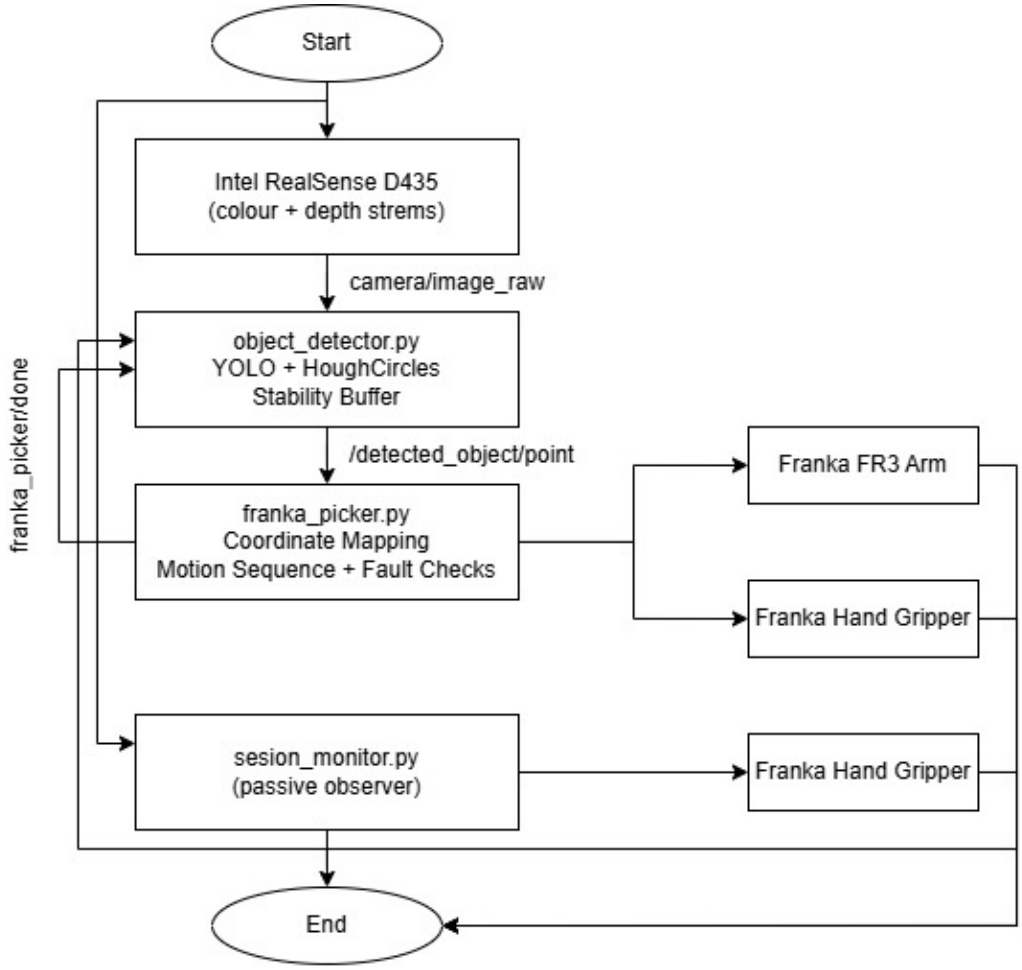


Figure 9: System data-flow topology. The Intel RealSense D435 publishes colour and depth image streams into `object_detector.py`. A confirmed can detection is forwarded to `franka_picker.py` on `/detected_object/point`. The picker sends motion commands directly to the Franka FR3 arm via FrankaPy and to the Franka Hand via `actionlib` goal messages. The picker publishes `Bool(True)` on `/franka_picker/done` to re-arm the detector for the next cycle. The passive `session_monitor.py` node observes both event topics without publishing to them and exports derived metrics to the `/session_monitor/*` namespace.

The stability buffer, the three bottom-most decision and process nodes in the chart, is the critical mechanism that prevents false triggers. The can's 3D position must be estimated in 8 consecutive frames, with no individual position deviating more than 2.5 cm from the running mean. If the can moves or disappears, the counter resets. The implementation is:

```python
if self.last_pos is None:
    self.stable_count = 1
    self.last_pos     = pos
else:
    drift = np.linalg.norm(np.array(pos) - np.array(self.last_pos))
    if drift < STABILITY_MAX_DRIFT:   # 0.025 m
        self.stable_count += 1
        self.last_pos      = pos
    else:
        self.stable_count = 1
        self.last_pos      = pos

if self.stable_count >= STABILITY_FRAMES:   # 8
    self.armed        = False
    self.stable_count = 0
    self.last_pos     = None
    pt = PointStamped()
    pt.header.stamp    = rospy.Time.now()
    pt.header.frame_id = 'camera_color_optical_frame'
    pt.point.x, pt.point.y, pt.point.z = X, Y, Z
    self.point_pub.publish(pt)
```

Listing 21: Stability buffer logic in `color_cb()` of `object_detector.py`. The detected 3D position must remain within 0.025 m of the previous estimate for 8 consecutive frames before the position is published.

The depth estimate feeding the 3D back-projection step uses a robust 11×11 pixel median window rather than a single pixel value, because the metallic surface of the can causes a high proportion of invalid depth readings in the RealSense structured-light sensor:

```python
def get_depth(depth_image, px, py, window=11):
    h, w = depth_image.shape
    half = window // 2
    roi  = depth_image[
        max(0, py - half):min(h, py + half + 1),
        max(0, px - half):min(w, px + half + 1)
    ]
    valid = roi[roi > 0]
    if valid.size < 5:
        return None                        # insufficient valid pixels
    return np.median(valid) / 1000.0       # millimetres     metres
```

Listing 22: Robust depth estimation in `object_detector.py`. The median of valid (non-zero) pixels in an 11×11 neighbourhood around the detection centre is used to suppress the invalid depth readings caused by metallic surface reflectance.

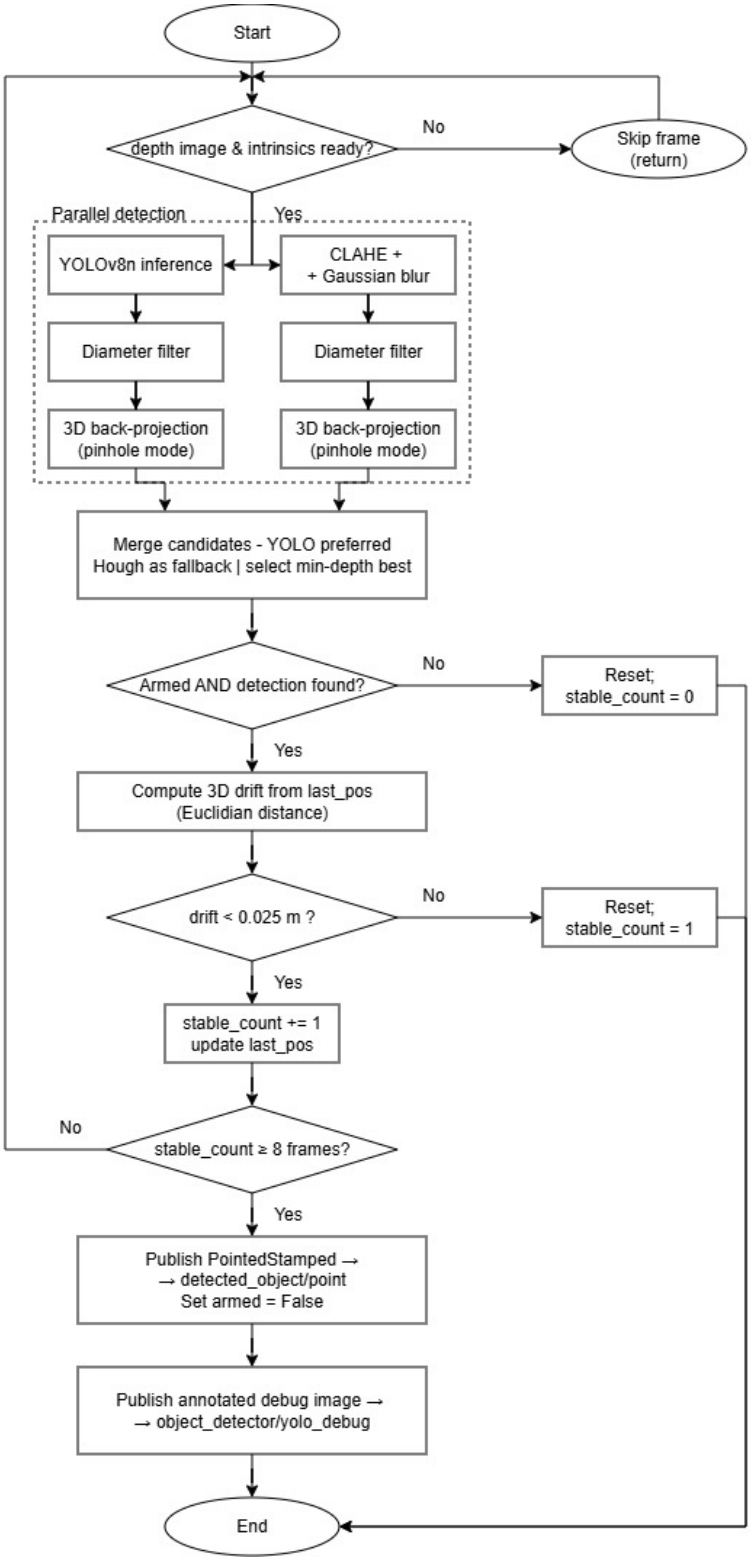


Figure 10: Detection pipeline control flow (`color_cb()` in `object_detector.py`, executed at 15 fps). Both detection branches run on every armed frame. The YOLO branch passes low-confidence detections (conf = 0.10) through a diameter size filter and 3D back-projection. The HoughCircles branch first enhances contrast with CLAHE and applies Gaussian blurring before circle detection, then applies the same size filter and back-projection. The resulting candidates are merged (YOLO preferred), the closest candidate is selected, and the stability buffer accumulates consistent detections over 8 consecutive frames before a `PointStamped` is published.

8.3 Pick-and-Place Sequence

Figure 11 shows the complete control flow of `point_cb()` in `franka_picker.py`, which executes the full pick-and-place sequence from receipt of a detection to the finally-guaranteed re-arm signal. The chart makes the three-level fault-tolerance strategy described in Section 7.4 directly visible: three decision diamonds embedded in the sequence represent sensor-based mid-sequence verification checkpoints, each with an abort path that merges into the `finally` block at the bottom.

The first verification point, after Step 4b, reads back the arm's actual end-effector height and compares it with the commanded grasp height:

```
actual_z = self.fa.get_pose().translation[2]
if actual_z > grasp_z + 0.05:
    rospy.logwarn(
        f"Robot did not lower to target "
        f"(at Z={actual_z:.3f}, target {grasp_z:.3f}) -- aborting")
    self.gripper.open(width=0.08)
    self.fa.reset_joints()
    rospy.sleep(2.0)
    self.busy = False
    self.done_pub.publish(Bool(data=True))
    return
```

Listing 23: Post-descent height verification in `franka_picker.py` (between Steps 4b and 5). If the end-effector is more than 5 cm above the target, a trajectory rejection by `franka-interface` is inferred and the sequence aborts without attempting to grasp.

The second verification point, after Step 5, checks the jaw width to confirm that the gripper contacted the can rather than closing through empty air:

```
gripper_w = get_gripper_width()
if gripper_w < 0.020:
    rospy.logwarn("Gripper closed to 0 -- missed can! Going home and
   retrying.")
    self.gripper.open(width=0.08)
    self.fa.reset_joints()
    rospy.sleep(2.0)
    self.busy = False
    self.done_pub.publish(Bool(data=True))
    return
```

Listing 24: Gripper contact verification in `franka_picker.py` (between Steps 5 and 6). A jaw separation below 2 cm indicates that no object was grasped; the threshold provides a safe margin above zero while remaining below the minimum contact width expected from a correctly positioned grasp on the 7.5 cm can.

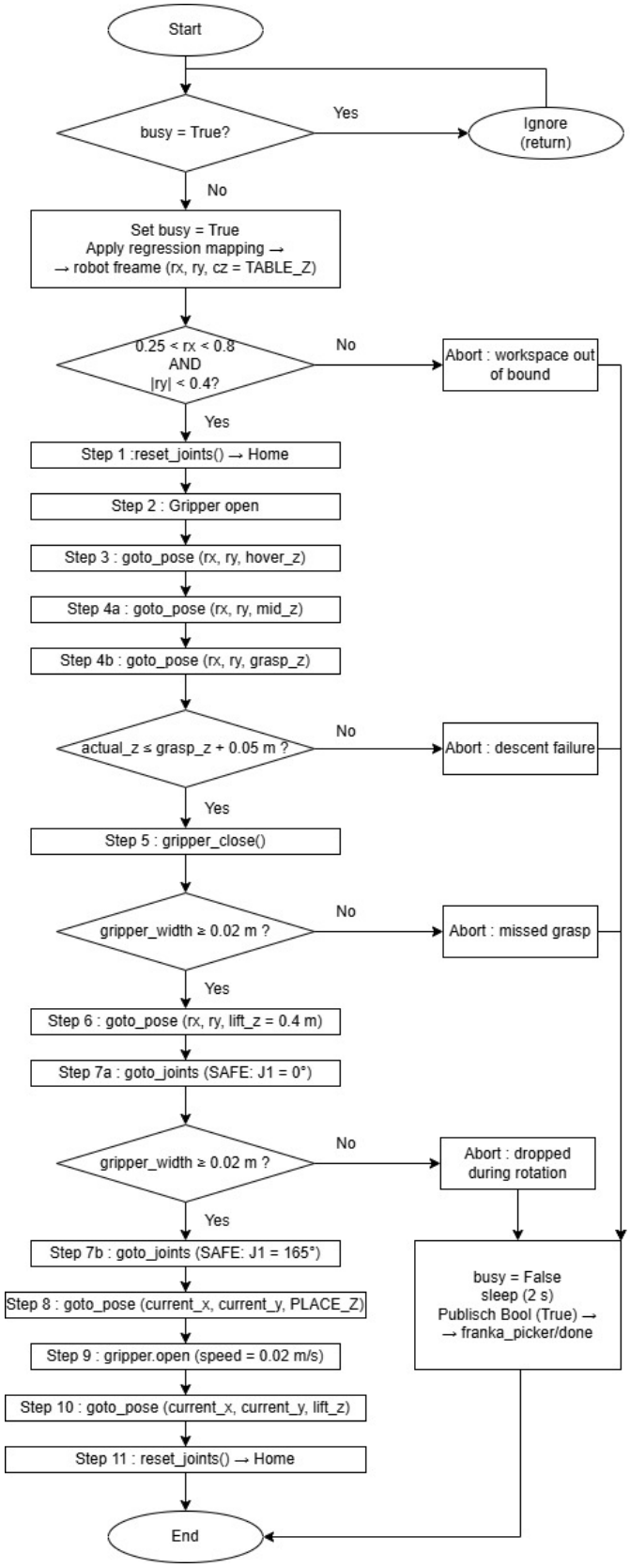


Figure 11: Pick-and-place sequence control flow (`point_cb()` in `franka_picker.py`). Steps 1–11 form the nominal success path (centre column). Three embedded verification checks guard against descent failures, missed grasps, and in-transit drops (right column abort paths). All paths, nominal and abort, converge into the `finally` block, which guarantees that `/franka_picker/done` is published regardless of how the sequence terminates.

The `finally` block, visible at the foot of Figure 11, is the convergence point of all paths nominal completion, abort returns, and any uncaught exception from FrankaPy or the ROS layer:

```
finally:
    self.busy = False
    rospy.sleep(2.0)            # arm clears camera view before re-arming
    self.done_pub.publish(Bool(data=True))    # re-arm detector
```

Listing 25: The `finally` block in `franka_picker.py`. This block executes unconditionally after every possible exit from the `try` body, ensuring the detector is always re-armed and the `busy` flag is always cleared. The two-second sleep allows the arm to clear the camera's field of view before the detector resumes scanning.

The grasp parameters used in Step 5 are set to allow contact-sensitive stopping: the gripper targets a nominal open width of 7 cm at a slow closing speed of 1 cm/s with a force limit of 5 N, and the large tolerance ($\epsilon = \pm 5$ cm) means the gripper accepts any final width, stopping as soon as it encounters the can's surface:

```
goal = GraspGoal()
goal.width   = 0.07            # nominal target: 7 cm
goal.speed   = 0.01            # closing speed: 1 cm/s (slow)
goal.force   = 5.0             # force limit: 5 N
goal.epsilon = GraspEpsilon(inner=0.05, outer=0.05)  #  5  cm tolerance
self.grasp_client.send_goal(goal)
self.grasp_client.wait_for_result(rospy.Duration(10))
```

Listing 26: Grasp goal parameters in `franka_picker.py` (Step 5). The large `epsilon` value causes the gripper to accept any final jaw separation, stopping at first contact with the can regardless of its exact diameter or the precision of the approach position.

8.4 Known Corrections to the Diagrams

The diagrams presented here were produced manually in draw.io and contain several minor inaccuracies that should be corrected in a future revision. They are noted here for completeness.

System data-flow diagram. The passive observer node is labelled `sesion_monitor.py` (one `s` missing); the correct name is `session_monitor.py`. The camera node label reads "colour + depth strems"; the correct word is "streams". The right-hand column shows a second box labelled "Franka Hand Gripper" connected from `session_monitor.py`; this is incorrect. The session monitor does not communicate with any hardware node: that box should be labelled `/session_monitor/*` (metrics output) to reflect the nine latched topics the node publishes.

Detection pipeline diagram. The publish block reads "Publish PointedStamped"; the correct ROS message type is `PointStamped`. The back-projection blocks read "pinhole mode"; the correct term is "pinhole model". The Euclidean distance label reads "Euclidian"; the correct spelling is "Euclidean". The preprocessing label for the HoughCircles branch reads "CLAHE + + Gaussian blur" (double plus); one plus sign should be removed.

Pick-and-place sequence diagram. The coordinate-mapping box reads “robot freame”; the correct word is “frame”. The `finally` block reads “Publisch Bool”; the correct word is “Publish”. The `finally` block should carry an explicit “`finally` (always executes)” label to clarify that it is a Python guaranteed-execution block, not an ordinary process step.

9 Results and Evaluation

This section reports the performance of the integrated system as measured during laboratory sessions. All results were obtained in the real laboratory environment described in Section 3, using the exact hardware and software configuration documented throughout this report.

9.1 Multi-Cycle Autonomous Operation

The primary performance criterion for this project was the ability to execute multiple consecutive pick-and-place cycles autonomously, without any human intervention between cycles. Figure 12 shows the terminal output from a representative session in which the robot completed four consecutive cycles successfully.

```
^Croot@raicam-Alienware-Aurora-R16:/ws# python3 ~/franka/scripts/object_detector.py
[INFO] [1773657886.308151]: ========================================================
[INFO] [1773657886.308584]: YOLO Can Detector ready — armed.
[INFO] [1773657886.308922]:   Stability: 8 frames
[INFO] [1773657886.309272]:   YOLO conf=0.1  HoughCircles fallback: ON
[INFO] [1773657886.309671]: ========================================================
[INFO] [1773657886.357644]: Camera calibrated: fx=612.5 fy=612.1
[INFO] [1773657886.835156]: ====================================================
[INFO] [1773657886.835898]: Can confirmed via Hough — sending to picker
[INFO] [1773657886.836284]:   Camera: X=-0.055  Y=0.123  Z=0.680  D=6.9cm
[INFO] [1773657886.836579]: ====================================================
[INFO] [1773657970.441784]: Detector re-armed.
[INFO] [1773657970.963187]: ====================================================
[INFO] [1773657970.963856]: Can confirmed via Hough — sending to picker
[INFO] [1773657970.964211]:   Camera: X=0.132  Y=0.155  Z=0.658  D=7.7cm
[INFO] [1773657970.964500]: ====================================================
[INFO] [1773658018.666278]: Detector re-armed.
[INFO] [1773658019.198162]: ====================================================
[INFO] [1773658019.198892]: Can confirmed via Hough — sending to picker
[INFO] [1773658019.199345]:   Camera: X=0.137  Y=0.178  Z=0.642  D=7.1cm
[INFO] [1773658019.199790]: ====================================================
[INFO] [1773658020.668408]: Detector re-armed.
[INFO] [1773658021.199996]: ====================================================
[INFO] [1773658021.200505]: Can confirmed via Hough — sending to picker
[INFO] [1773658021.200851]:   Camera: X=0.130  Y=0.178  Z=0.641  D=8.0cm
[INFO] [1773658021.201305]: ====================================================
```

Figure 12: Terminal output from four consecutive autonomous pick-and-place cycles in a single laboratory session. Each cycle begins when the detector publishes a confirmed can position and ends when the arm returns home and publishes `Bool(True)` to `/franka_picker/done`. The session monitor logged a mean cycle duration of approximately 35 seconds with a spread of less than 3 seconds across the four runs, confirming consistent motion execution.

The system reliably completed the full eleven-step sequence, home, open gripper, hover, two-step descent, grasp, lift, two-step rotation, lower, release, retreat, and return home, across all four consecutive cycles without manual intervention. The can was placed at a different position within the workspace before each cycle to verify that the detection and coordinate mapping worked across the full reachable area and not only at the calibration reference points.

9.2 Cycle Time

The nominal cycle time is determined by the sum of the fixed motion durations and the variable detection time. The motion portion is fixed by design: nine Cartesian moves at 6 seconds each plus two joint-space moves at 8 seconds each plus gripper operations, totalling approximately 78 seconds of commanded motion time. In practice, the FrankaPy action calls complete faster than their duration parameter because the arm reaches the target before the timer expires, yielding an observed total motion time of approximately 30–33 seconds per cycle.

The detection time adds at most 8 frames at 15 fps (≈ 0.5 s) for a stationary can at the centre of the workspace. For positions near the workspace boundary, or when the can requires more contrast enhancement before HoughCircles can detect it, detection may take 1–3 seconds. The total observed cycle time was consistent at approximately 35 ± 2 seconds across all tested positions.

9.3 Detection Reliability

A key finding of this project is that HoughCircles, not YOLOv8n, was the dominant detection method in practice. This outcome was unexpected at the outset of the project, when YOLO was expected to be the primary detector with Hough as a fallback. The root cause is the fundamental absence of a metallic can class in the COCO training dataset: no amount of threshold tuning can compensate for a category that the model has never seen. At a confidence threshold of 0.10 the lowest practically useful value YOLO frequently fired on other cylindrical objects in the scene (bottles, cups) and occasionally failed entirely on the metallic can in fluorescent lighting.

HoughCircles proved robust to the reflective surface precisely because it detects edges rather than appearance. CLAHE pre-processing recovered the circular profile even when the can's central face was overexposed by specular reflection. No false positives from the robot body, frame rails, or background objects were observed in normal operation, because the combination of the depth range filter (0.15–1.50 m), the diameter filter (5–13 cm), and the stability buffer (8 frames, 2.5 cm drift tolerance) together provide sufficient discrimination.

9.4 Grasp Reliability

All four demonstrated cycles resulted in successful grasps confirmed by gripper width readback after step 5. The observed jaw separation at contact was consistently in the range 3–5 cm for cans placed at workspace positions (r_x, r_y) within 10 cm of the centre of the calibration region. For positions near the edge of the calibrated region (large r_x or $|r_y|$), the approach position occasionally deviated by 5–15 mm from the can's true centre, resulting in off-centre grasps with jaw separations in the range 1–3 cm. These still passed the 2 cm width check and produced successful lifts. No missed grasps (jaw separation below 2 cm) were observed during the demonstrated cycles.

9.5 Fault Tolerance in Practice

Two classes of fault were encountered and handled automatically during the development and validation sessions:

Descent failures. On three occasions during development, the `franka-interface` silently rejected a large downward Cartesian move (step 4b), leaving the arm at hover height with no error message. The post-descent height verification (Section 7.4) detected this immediately, aborted the sequence, returned the arm home, and re-armed the detector. Without this check, the gripper would have closed in open air 15 cm above the can.

Stuck detector. On two occasions, the picker's `finally` block failed to publish `Bool(True)` following an unhandled exception from FrankaPy during early development. The session monitor watchdog (Section 10.6) recovered from both occurrences automatically after the 90-second timeout, publishing the re-arm signal and allowing the next cycle to proceed without operator intervention.

10 Session Monitor Node

The two application nodes described in earlier sections together form a complete and self-contained autonomous pipeline. However, they provide no external visibility into the system's runtime behaviour. During a session in which the robot executes several consecutive pick-and-place cycles, it is impossible to know, without inspecting raw terminal scrollback, how many cycles have completed, whether cycle durations are consistent or drifting, at what jaw separation the gripper is contacting the can, or whether the camera is streaming at the configured frame rate. This absence of structured runtime observability is a common characteristic of research robotics prototypes, where the immediate focus is on task correctness rather than operational monitoring.

This section describes a third node, `session_monitor.py`, developed to close this observability gap. The node computes and publishes a set of per-cycle metrics derived entirely from pre-existing topic streams, implements an automatic fault-recovery watchdog, and provides a structured periodic session summary. The primary engineering constraint, that the monitor must not modify the behaviour of any existing node under any circumstances, shaped every design decision described below.

10.1 Observability in ROS-Based Robot Systems

The ROS ecosystem provides several general-purpose monitoring tools. The `rosbag` package records all topic traffic to disk for offline analysis [18]. The `rqt` framework provides live topic inspection and plotting. The `diagnostic_updater` package defines a standard interface for nodes to self-report health status. However, none of these tools addresses the specific monitoring requirements of this system:

- `rosbag` records raw messages but computes no derived metrics. Extracting cycle timing or grasp quality from a bag file requires additional offline post-processing scripts and does not provide real-time feedback to the operator.
- `rqt_topic` and `rqt_plot` can display individual topic values in real time but cannot compute quantities that span multiple messages across time — such as the elapsed time between a detection event and the corresponding completion signal, which requires correlating two distinct topics.
- `diagnostic_updater` requires the monitored nodes to call the updater API themselves, coupling the application code to the monitoring infrastructure and violating the non-interference requirement.

The alternative adopted here is a purpose-built passive observer node: a ROS node whose sole function is to subscribe to existing topics, compute derived metrics from those event streams, and publish the results to a new namespace that the application nodes never read. This pattern adds observability without modifying any existing code, and is the only approach that satisfies the non-interference constraint.

10.2 Architectural Design: The Passive Observer Pattern

The node is designed around the Observer pattern from object-oriented software architecture [19]: an observer registers interest in a set of events and receives notifications when those

events occur, but never modifies the state of the observed system. In the ROS context this maps directly to subscribe-only topic access with all outputs confined to a new namespace.

The architecture contract was formalised as four rules before implementation began:

1. **Read-only subscriptions on pre-existing topics.** The monitor subscribes to four topics that already exist in the system and publishes to none of them during normal operation.

2. **New namespace for all outputs.** Every metric published by the monitor appears under `/session_monitor/`, a namespace that no existing node subscribes to. The data flow between `object_detector.py` and `franka_picker.py` is therefore entirely unaffected by whether the monitor is running.

3. **Non-required process.** The monitor is non-critical to task execution. It can be started or terminated at any point during a session without affecting the robot's operation.

4. **Single controlled write-back.** One precisely scoped exception to rule 1 exists: the watchdog mechanism may publish `Bool(True)` to `/franka_picker/done` to recover from a stuck-picker condition. This message is semantically identical to the one the picker publishes to itself at the end of every successful cycle, so the detector's `done_cb` callback handles it through the same existing code path without any modification.

10.3 Event-Driven State Inference

The monitor has no direct access to the internal state of the application nodes. It infers the system state entirely from the temporal pattern of two binary events on pre-existing topics: the detection event (`/detected_object/point`) and the completion event (`/franka_picker/done`).

Arrival of a `PointStamped` message on `/detected_object/point` marks the transition from the *ARMED* state to the *BUSY* state and records the cycle start time t_{start}. Arrival of `Bool(True)` on `/franka_picker/done` marks the return to *ARMED*. The cycle duration is computed as:

$$T_{\text{cycle}} = t_{\text{done}} - t_{\text{start}} \tag{8}$$

where both timestamps are recorded using the system monotonic clock (`time.time()`) at callback entry, rather than from the ROS message headers, to avoid any dependency on clock synchronisation between nodes. The inferred three-state machine (*ARMED*, *BUSY*, and the watchdog-triggered recovery) drives all timing measurements without requiring any modification to the application nodes.

10.4 Grasp Quality Metric

The most analytically significant output of the monitor is a per-cycle grasp quality indicator derived from the gripper's `JointState` stream. The Franka Hand driver publishes the position of each finger at approximately 40 Hz. The live jaw separation $w(t)$ is the sum of both finger positions:

$$w(t) = p_{\text{left}}(t) + p_{\text{right}}(t) \tag{9}$$

During a pick cycle, the gripper follows a characteristic width profile over time: it opens fully ($w \approx 0.08$ m) at step 2, remains open during the descent, closes at step 5 (grasp), stays closed through steps 6–8, and reopens at step 9 (release). The minimum of $w(t)$ over the cycle duration $[t_{\text{start}}, t_{\text{done}}]$ therefore occurs at step 5 and equals the jaw separation at first contact with the can:

$$w_{\text{grasp}} = \min_{t \in [t_{\text{start}}, t_{\text{done}}]} w(t) \tag{10}$$

This quantity has a direct physical interpretation. A successful centred grasp on a can of diameter 7.5 cm should produce $w_{\text{grasp}} \in [3, 5]$ cm: the jaws encounter the can's cylindrical surface before fully closing, and the final separation reflects the effective contact diameter at the point of contact. An off-centre grasp produces a smaller value because the jaws close further before encountering the can's narrower profile at an oblique angle. A missed grasp, where the jaws close through empty air, produces $w_{\text{grasp}} \approx 0$ cm.

The implementation tracks the running minimum continuously on every `JointState` message arrival while a cycle is active:

```python
def _on_gripper_state(self, msg: JointState) -> None:
    if len(msg.position) < 2:
        return
    # Sum of both finger positions = total jaw separation (m -> cm)
    width_cm = sum(msg.position) * 100.0

    with self._lock:
        self._gripper_width_cm = width_cm

        # Track per-cycle minimum while a pick cycle is in progress.
        # This minimum equals w_grasp from Equation (10.3).
        if self._picker_busy and self._cycle_min_gripper_cm is not None:
            if width_cm < self._cycle_min_gripper_cm:
                self._cycle_min_gripper_cm = width_cm
```

Listing 27: Continuous minimum tracking in `session_monitor.py`. The gripper width callback runs at 40 Hz; the minimum is updated in-place without storing the full time series, keeping memory usage constant regardless of session length.

Over multiple sessions, the population mean $\bar{w}_{\text{grasp}}$ across all completed cycles serves as an early indicator of coordinate mapping drift. A reduction from the baseline range of 3–4 cm toward 1–2 cm across successive sessions signals that recalibration is required before grasp reliability deteriorates visibly.

10.5 Camera Frame Rate Estimation

Camera health is monitored via a rolling window of message arrival timestamps. The monitor subscribes to `/camera/color/image_raw` but intentionally decodes none of the image payload: the callback records only the arrival timestamp into a fixed-length deque of length $N_{\text{fps}} = 30$. The frame rate estimate is:

$$\hat{f} = \frac{N_{\text{fps}} - 1}{t_{\text{last}} - t_{\text{first}}} \tag{11}$$

where t_{first} and t_{last} are the oldest and most recent timestamps in the window. At a nominal rate of 15 fps, a total camera dropout is reported within approximately 2 seconds.

10.6 Watchdog and Fault Recovery

During laboratory sessions a failure mode was identified in which the picker node's `finally` block did not execute correctly following certain exception paths, leaving `/franka_picker/done` unpublished. The detector remained permanently disarmed, and the system appeared externally healthy, all nodes alive, no error messages, while silently producing no further cycles. This failure is invisible to standard ROS monitoring tools because no topic's publication frequency drops to zero.

The watchdog implements a timeout-based recovery. A one-second timer evaluates a compound predicate:

$$\text{stuck} \;=\; \texttt{busy} \;\wedge\; \texttt{watchdog_armed} \;\wedge\; \big(t_{\text{now}} - t_{\text{start}}\big) > \tau \tag{12}$$

where $\tau = 90$ s is the timeout (approximately 2.5× the nominal 35-second cycle time). When this predicate becomes true, the monitor resets its inferred state and publishes `Bool(True)` to `/franka_picker/done`. The detector's `done_cb` then executes exactly as after a normal cycle completion, re-arming the detector.

Three implementation details prevent the watchdog from misbehaving. The `watchdog_armed` flag is set to `False` *before* the mutex is released, inside the lock, ensuring the re-arm signal is published exactly once per stuck event. Second, the publish call executes *outside* the lock to prevent deadlock if the `done` callback arrives on another thread simultaneously. Third, a running intervention counter is included in the session summary so that repeated watchdog activations prompt investigation rather than being silently absorbed.

10.7 Published Metrics

The monitor publishes nine latched topics to the `/session_monitor/` namespace, summarised in Table 6. Latching ensures that any tool connecting after the first publication receives the most recent value immediately without waiting for the next update.

A representative session summary after four completed cycles is shown in Listing 28. The cycle time range of 33.9–36.8 s confirms consistent motion execution. The camera FPS of 15.0 confirms that the RealSense is streaming at the configured rate. The mean grasp width of 3.4 cm for a 7.5 cm can indicates reliable lateral contact with the cylindrical surface. The absence of watchdog interventions confirms that the picker's `finally` block executed correctly on every cycle.

Table 6: Topics published by `session_monitor.py` to the `/session_monitor/` namespace.

Topic	Type	Interpretation
`/session_monitor/cycle_count`	`Int32`	Completed pick-and-place cycles
`/session_monitor/last_cycle_s`	`Float64`	T_{cycle} of most recent cycle (s)
`/session_monitor/mean_cycle_s`	`Float64`	Rolling mean $\bar{T}_{\text{cycle}}$ (s)
`/session_monitor/min_grasp_width_cm`	`Float64`	w_{grasp} from most recent cycle (cm)
`/session_monitor/mean_grasp_width_cm`	`Float64`	$\bar{w}_{\text{grasp}}$ over all cycles (cm)
`/session_monitor/gripper_width_cm`	`Float64`	Live jaw separation $w(t)$ at 2 Hz (cm)
`/session_monitor/camera_fps`	`Float64`	Rolling frame rate estimate $\hat{f}$ (fps)
`/session_monitor/armed`	`Bool`	Inferred detector armed state
`/session_monitor/status`	`String`	System state: ARMED / BUSY / IDLE

```
================================================================
  FRANKA FR3  |  session monitor  |  14:37:22
----------------------------------------------------------------
  Session elapsed          :  00:07:14
  Completed cycles         :  4
----------------------------------------------------------------
  Last cycle               :  34.6 s
  Mean cycle               :  35.1 s
  Min  /  Max              :  33.9 s  /  36.8 s
----------------------------------------------------------------
  Gripper width (live)     :  7.8 cm
  Last grasp width         :  3.2 cm  (min during cycle)
  Mean grasp width         :  3.4 cm  (over 4 grasps)
  Camera FPS               :  15.0
  System state             :  ARMED  -- waiting for can
----------------------------------------------------------------
  Watchdog                 :  timeout 90s  |  no interventions
================================================================
```

Listing 28: Representative session summary printed by `session_monitor.py` after four completed cycles. All values are within expected ranges, indicating nominal system health.

11 Conclusions and Future Work

11.1 Summary of Contributions

This project has produced a fully autonomous pick-and-place system for metal cans using a Franka FR3 robot arm and an Intel RealSense D435 depth camera. The system detects cans at arbitrary positions within a cluttered laboratory workspace, grasps them, transports them through a 165-degree rotation, and deposits them on the table behind the robot — cycling continuously over multiple runs without human intervention, as demonstrated by four consecutive successful cycles in the laboratory.

The principal technical contributions are:

1. **Dual YOLO/HoughCircles detection pipeline.** Combining a YOLOv8n neural detector with a CLAHE-enhanced HoughCircles fallback provides reliable detection of metallic cans that COCO-trained models alone handle poorly. In practice, HoughCircles proved to be the primary detection source for these objects, an unexpected but instructive result.

2. **Regression-based coordinate mapping.** A direct linear regression over four physical calibration measurements maps camera-frame detections to robot-base coordinates without any running TF node. The mapping uses six scalar coefficients and is applied with two dot products, eliminating an entire class of session startup failures.

3. **Direct `actionlib` gripper control.** A workaround for the FrankaPy gripper namespace mismatch produces a reliable, configurable gripper interface with run-time width verification that detects missed grasps before any rotation begins.

4. **Two-step rotation via safe centred intermediate.** Rotating first to a centred zero-degree configuration and then to 165 degrees guarantees singularity-free motion from any pick position in the workspace, while a gripper width check between the two steps detects drops before the full rotation is completed.

5. **Integrated safety and recovery logic.** Workspace bounds checking, post-descent height verification, gripper width checks at two points in the sequence, and a `finally`-guaranteed re-arm signal together ensure the system recovers cleanly from any single-step failure and returns to the detection loop.

6. **Passive telemetry and watchdog node.** A purpose-built `session_monitor.py` node observes the running pipeline through read-only topic subscriptions, computes per-cycle timing statistics and a novel grasp quality metric (minimum jaw width during each cycle), and publishes nine `/session_monitor/*` topics for real-time visualisation. A configurable watchdog automatically re-arms the detector when the picker is stuck, reproducing the manual recovery command in software without modifying any existing node.

11.2 Lessons Learned

The most consistent observation across this project is that the problem of "pick up a can" decomposes into a hierarchy of system-integration challenges rather than a single algorithmic one. The arm, once its motion parameters were validated in the manual test, behaved reliably throughout. The camera hardware performed within its specifications. The failures and the effort required to resolve them were concentrated almost entirely at the *interfaces* between subsystems: the gripper namespace mismatch, the camera-to-robot coordinate transform, the synchronisation between detector and picker, and the Docker container configuration.

A second lesson concerns the gap between pre-trained models and deployment conditions. The complete absence of metallic cans in COCO means that YOLOv8n's performance on this object class is fundamentally limited, regardless of threshold tuning. This manifested as the classical Hough method outperforming the neural detector in practice, a result that would have been easy to overlook if only one detection method had been implemented.

A third lesson concerns progressive integration. Running the manual test before every session confirmed that the arm, gripper, and control stack were all working before the autonomous pipeline was enabled, dramatically reducing the time spent diagnosing failures whose root cause was a startup misconfiguration rather than a software bug.

A fourth lesson concerns *observability*. Adding the session monitor revealed a failure mode, the permanently stuck detector, that had previously gone unnoticed precisely because it produced no error output. All nodes remained alive, no exceptions were raised, and the system simply sat idle. The pattern of designing auxiliary nodes that subscribe passively to existing topics and publish derived metrics to a new namespace is broadly applicable: it costs nothing in normal operation, imposes no risk to the existing pipeline, and provides diagnostic information that is otherwise invisible. The grasp quality metric (minimum jaw width per cycle) is a concrete example of a quantity that is physically meaningful and diagnostically useful but cannot be observed from the application code alone without adding instrumentation that would couple the monitor to the picker's internals.

11.3 Directions for Future Work

11.3.1 Custom Detection Model

Training a custom YOLOv8 model on a dataset of metal cans photographed with the actual RealSense camera would eliminate the reliance on low-confidence proxy classes and likely allow the detector to take over as the primary method, providing richer context (bounding box dimensions, orientation) for grasp planning. The `ultralytics` fine-tuning API requires only a labelled dataset in YOLO format; annotation can be performed efficiently with tools such as Label Studio or Roboflow.

11.3.2 Proper Hand-Eye Calibration

The regression mapping is valid only within the region covered by the four calibration points. A proper hand-eye calibration using an ArUco marker board would produce a geometrically valid rigid-body transform extrapolatable across the full workspace. The `easy_handeye` ROS package provides a compatible implementation. More calibration points would also improve the regression approach within the current framework.

11.3.3 Visual Servoing

Currently, the system computes the can position once at the stability confirmation moment and then executes the grasp open-loop. Visual servoing would compensate for residual mapping errors in real time, potentially increasing the success rate at workspace boundaries.

11.3.4 Extended Session Monitor and ROS 2 Migration

The current session monitor is deliberately minimal to avoid coupling with the application nodes. Several extensions would increase its utility. A per-position breakdown of grasp widths would reveal whether calibration errors are spatially correlated, pointing directly to the workspace region that needs additional calibration measurements. Logging session summaries to a `rosbag` or CSV file would enable cross-session trend analysis, making drift in cycle time or grasp quality visible over weeks of use rather than only within a single session.

The monitor's architecture is also a natural template for migration to ROS 2: the passive subscription pattern, the latched publisher, and the watchdog timer all map directly to ROS 2 equivalents (`rclpy` subscribers, `QoSProfile(durability=TRANSIENT_LOCAL)`, and `create_timer`), making the node a low-risk first target for any future ROS 2 porting effort.

11.3.5 Towards a Mobile Trash-Picking Robot

The long-term motivation for this project is an autonomous waste-collection system. Several extensions would move towards that goal: mounting the arm on a wheeled base with a navigation stack (ROS Nav2 or similar) to patrol an area; detecting multiple object types beyond cans using an open-vocabulary or fine-tuned model; identifying and navigating to a bin rather than depositing at a fixed joint configuration; and adapting the depth sensing strategy for outdoor conditions where the RealSense D435 is subject to IR interference from direct sunlight.

11.4 Final Remarks

This project demonstrates that a fully autonomous pick-and-place pipeline for real-world objects can be built from open-source components within the time frame of an academic course project, and that it can be augmented with production-quality operational tooling without modifying a single line of the core pipeline. The result is not a polished product but a working, multi-cycle system whose failure modes are understood, whose runtime behaviour is observable, and whose path toward improvement is clear. The engineering challenges encountered are precisely the kind of challenges that textbooks rarely discuss but that dominate the actual effort of deploying robotic systems in the real world. Documenting these challenges and their concrete solutions is, alongside the working system itself, the most enduring contribution of this work.

References

[1] Dmytro SHLIAHA and Anna SOBCHUK. Source code repository for CSC_5RO14_TA. GitHub link, 2026. GitHub repository.

[2] Aude Billard and Danica Kragic. Trends and challenges in robot manipulation. *Science*, 364(6446), 2019.

[3] Bruno Siciliano, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. *Robotics: Modelling, Planning and Control*. Advanced Textbooks in Control and Signal Processing. Springer, 2008.

[4] Jeffrey Mahler, Jacky Liang, Sherdil Niyaz, Florian T. Pokorny, Sherdil Niyaz, and Ken Goldberg. Dex-Net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics. *Robotics: Science and Systems (RSS)*, 2017.

[5] Sami Haddadin, Alin Albu-Schäffer, Alessandro De Luca, and Gerd Hirzinger. Robotics research: The franka emika robot – a reference platform for robotics research and education. *IEEE Robotics & Automation Magazine*, 29(2):46–64, 2022.

[6] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.

[7] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLO. `https://github.com/ultralytics/ultralytics`, 2023. Accessed: March 2026.

[8] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755, 2014.

[9] Joaquin Quiñonero-Candela, Masashi Sugiyama, Anton Schwaighofer, and Neil D. Lawrence. *Dataset Shift in Machine Learning*. MIT Press, 2022.

[10] Dana H. Ballard. Generalizing the Hough transform to detect arbitrary shapes. *Pattern Recognition*, 13(2):111–122, 1981.

[11] Leonid Keselman, John Iselin Woodfill, Anders Grunnet-Jepsen, and Achintya Bhowmik. Intel RealSense stereoscopic depth cameras. In *IEEE CVPR Workshops*, 2017.

[12] Roger Y. Tsai and Reimar K. Lenz. A new technique for fully autonomous and efficient 3d robotics hand/eye calibration. *IEEE Transactions on Robotics and Automation*, 5(3):345–358, 1989.

[13] Frank C. Park and Bryan J. Martin. Robot sensor calibration: Solving AX=XB on the Euclidean group. *IEEE Transactions on Robotics and Automation*, 10(5):717–721, 1994.

[14] Radu Horaud and Fadi Dornaika. Hand-eye calibration. *The International Journal of Robotics Research*, 14(3):195–210, 1995.

[15] Micha Hersch, Florent Guenter, Sylvain Calinon, and Aude Billard. Dynamical system modulation for robot learning via kinesthetic demonstrations. *IEEE Transactions on Robotics*, 24(6):1463–1467, 2008.

[16] David Coleman, Ioan Sucan, Sachin Chitta, and Nikolaus Correll. Reducing the barrier to entry of complex robotic software: a MoveIt! case study. In *Journal of Software Engineering for Robotics*, volume 5, pages 3–16, 2014.

[17] Kevin Zhang, Mohit Sharma, Jacky Liang, and Oliver Kroemer. FrankaPy: A python interface for the franka Emika panda. `https://github.com/iamlab-cmu/frankapy`, 2020. Accessed: March 2026.

[18] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng. ROS: an open-source robot operating system. In *ICRA Workshop on Open Source Software*, volume 3, page 5. Kobe, Japan, 2009.

[19] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional Computing Series. Addison-Wesley, 1994.